UNIVERSIDADE PAULISTA — UNIP

Câmpus Santos, Rangel — SP

Documentação de Atividades Práticas Supervisionadas (APS)

VEM AÍ — SISTEMA DISTRIBUÍDO DE AVISO DE CHEGADA DO TRANSPORTE PÚBLICO DA BAIXADA SANTISTA

Desenvolvimento de uma aplicação de sistema distribuído para dispositivo móvel

7º / 8º Ciência da Computação (CC) — Desenvolvimento de Sistemas Distribuídos — DSD

Revisão 03

Christyan Charlles — R012CJ9

Daniele dos Santos França — F356FG7

Enzo Santos Pita — N0885D0

Luiz Santana de Araujo — N047HF6

Welber Willian da Silva — F352951

Santos — SP

2026

Christyan Charlles — R012CJ9
Daniele dos Santos França — F356FG7
Enzo Santos Pita — N0885D0
Luiz Santana de Araujo — N047HF6
Welber Willian da Silva — F352951

VEM AÍ — SISTEMA DISTRIBUÍDO DE AVISO DE CHEGADA DO TRANSPORTE PÚBLICO DA BAIXADA SANTISTA

Trabalho apresentado à disciplina de Atividades Práticas Supervisionadas do curso de Ciência da Computação da Universidade Paulista — UNIP, Câmpus Santos, como requisito parcial para avaliação semestral. Disciplina vinculada: Desenvolvimento de Sistemas Distribuídos — DSD. Professor supervisor: Vinícius Eduardo Ferreira dos Santos Silva.

Santos — SP
2026

LISTA DE FIGURAS

Figura 1 – Arquitetura geral do sistema.....29
Figura 2 – Implantação em produção.....31
Figura 3 – Montagem do catálogo de pontos pelo coletor.....32
Figura 4 – Busca de pontos com espalhamento e coleta.....33
Figura 5 – Sequência de criação de um alerta.....34
Figura 6 – Ciclo do monitor.....35
Figura 7 – Regras de notificação.....36
Figura 8 – Resposta da central à falha de coletores.....38
Figura 9 – Modelo de dados da central.....39
Figura 10 – Diagrama de casos de uso.....41
Figura 11 – Tela inicial (a) e pontos próximos ao usuário (b).....46
Figura 12 – Resultado da busca (a) e previsões do ponto 9306 (b).....47
Figura 13 – Veículo no mapa da linha (a) e criação de alerta (b).....48
Figura 14 – Mapa da linha com veículos ao vivo (a) e tabela horária (b).....49
Figura 15 – Tela de alertas do dispositivo.....50

LISTA DE QUADROS

Quadro 1 – Recursos dos portais utilizados na coleta.....22
Quadro 2 – Requisitos funcionais.....23
Quadro 3 – Requisitos não funcionais.....23
Quadro 4 – Tecnologias selecionadas.....24
Quadro 5 – Comparação entre as revisões do plano.....25
Quadro 6 – Riscos e mitigações.....26
Quadro 7 – Cronograma de desenvolvimento.....27
Quadro 8 – Divisão de tarefas.....27
Quadro 9 – Responsabilidades dos nós.....29
Quadro 10 – Exemplo de funcionamento do monitor.....37
Quadro 11 – Comportamento diante de falhas.....38
Quadro 12 – Entidades do modelo de dados.....39
Quadro 13 – Rotas da interface de programação da central.....40
Quadro 14 – Módulos do servidor.....42
Quadro 15 – Glossário de termos.....55

LISTA DE CÓDIGOS

Código 1 – Regras de aproximação e de desaparecimento.....42
Código 2 – Espalhamento e coleta na central.....43
Código 3 – Ciclo do monitor.....43
Código 4 – Registro idempotente de notificação.....44
Código 5 – Análise do quadro de previsões.....44
Código 6 – Recebimento e toque da notificação no service worker.....45

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface* (interface de programação de aplicações)
APS	Atividades Práticas Supervisionadas
CSS	*Cascading Style Sheets*
DSD	Desenvolvimento de Sistemas Distribuídos
EMTU	Empresa Metropolitana de Transportes Urbanos de São Paulo
GPS	*Global Positioning System*
HTML	*HyperText Markup Language*
HTTP	*Hypertext Transfer Protocol*
HTTPS	*Hypertext Transfer Protocol Secure*
IBGE	Instituto Brasileiro de Geografia e Estatística
JSON	*JavaScript Object Notation*
LGPD	Lei Geral de Proteção de Dados Pessoais
PWA	*Progressive Web App* (aplicação web progressiva)
REST	*Representational State Transfer*
RF	Requisito funcional
RFC	*Request for Comments*
RMBS	Região Metropolitana da Baixada Santista
RNF	Requisito não funcional
SPA	*Single-Page Application* (aplicação de página única)
SQL	*Structured Query Language*
TLS	*Transport Layer Security*
UNIP	Universidade Paulista
VAPID	*Voluntary Application Server Identification*
VPS	*Virtual Private Server* (servidor virtual privado)

SUMÁRIO

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO.....8
1.1 Objetivos específicos.....8
1.2 Motivação.....9
2 INTRODUÇÃO.....11
2.1 Contexto regional.....11
2.2 Sistemas de informação ao usuário.....12
2.3 A proposta.....12
2.4 Justificativa da arquitetura distribuída.....13
2.5 Estrutura do documento.....13
3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS.....15
3.1 Sistemas distribuídos: conceitos gerais.....15
3.1.1 Transparência.....15
3.1.2 Escalabilidade.....16
3.1.3 Modelos de comunicação.....16
3.2 Detecção e tolerância a falhas.....17
3.3 Idempotência e entrega de mensagens.....17
3.4 Concorrência assíncrona.....18
3.5 Virtualização por contêineres.....18
3.6 Comunicação iniciada pelo servidor: Web Push.....19
3.7 Aplicações web progressivas como plataforma móvel.....19
3.8 Extração de dados da web.....20
4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO.....21
4.1 Descrição do problema.....21
4.2 Escopo.....21
4.3 Fontes de dados.....22
4.3.1 Conduta de coleta.....22
4.4 Requisitos funcionais.....23
4.5 Requisitos não funcionais.....23
4.6 Tecnologias selecionadas.....24
4.6.1 Aplicação web progressiva como plataforma móvel.....24
4.7 Revisão do plano em relação à Revisão 02.....25
4.8 Análise de custos e de escalabilidade.....26

4.9 Riscos identificados.....26
4.10 Cronograma.....27
4.11 Divisão de tarefas.....27
5 PROJETO DO PROGRAMA.....29
5.1 Arquitetura geral.....29
5.2 Implantação.....30
5.3 Módulo coletor.....31
5.4 Central e agregação regional.....33
5.5 Criação de alertas.....34
5.6 Monitor e regras de notificação.....35
5.7 Tratamento de falhas parciais.....37
5.8 Modelo de dados.....39
5.9 Interface de programação.....40
5.10 Interface do usuário.....40
5.11 Diagrama de casos de uso.....41
6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA.....42
7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO.....46
8 INTERDISCIPLINARIDADE E CONTRIBUIÇÃO PARA A FORMAÇÃO.....51
8.1 Áreas do conhecimento mobilizadas.....51
8.2 Contribuição para a formação dos integrantes.....52
9 CONCLUSÃO.....53
GLOSSÁRIO.....55
REFERÊNCIAS.....56

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO

Este trabalho tem como objetivo geral desenvolver uma aplicação baseada em sistemas distribuídos, destinada a dispositivos móveis, que avisa o passageiro do transporte público coletivo da Região Metropolitana da Baixada Santista quando o ônibus que ele costuma utilizar se aproxima do ponto de embarque, ou quando esse veículo deixa de ser rastreado. A aplicação, denominada **Vem Aí**, reúne em uma única interface as informações publicadas pelas duas operadoras que atendem a região e entrega os avisos por meio de notificações enviadas ao dispositivo, sem que o usuário precise manter a aplicação aberta.

O tema atende à proposta da disciplina de Atividades Práticas Supervisionadas para o semestre, que solicita o desenvolvimento de uma aplicação distribuída voltada ao gerenciamento de informações ambientais urbanas, entre as quais se inclui expressamente o trânsito e o transporte público. A escolha do transporte coletivo como recorte justifica-se por sua relevância social direta e por sua relação com a qualidade ambiental urbana, uma vez que a eficiência percebida do transporte público influencia as decisões de deslocamento da população e, por consequência, a emissão de poluentes e a intensidade do congestionamento viário.

Esta revisão do documento (Revisão 03) substitui a Revisão 02, que descrevia uma solução executada em nós de borda de uma plataforma de computação em nuvem e escrita em TypeScript. Ao longo do desenvolvimento, o grupo reorientou o projeto para uma arquitetura de serviços escritos em Python e executados em contêineres, com escopo concentrado no aviso de chegada. As razões dessa mudança são expostas na seção 4.7.

1.1 Objetivos específicos

Para a consecução do objetivo geral, foram estabelecidos os seguintes objetivos específicos:

a) projetar e implementar uma arquitetura distribuída composta por processos independentes (front-end, central e coletores), que se comunicam exclusivamente por mensagens HTTP com conteúdo em formato JSON;

b) desenvolver coletores capazes de obter, por extração de dados da web, as previsões de chegada, os pontos de parada, os itinerários, as tabelas horárias e a posição dos veículos publicados nos portais das operadoras;

c) unificar as respostas dos coletores em uma visão única da região, de modo que o usuário nunca precise escolher a operadora;

d) implementar um monitor que acompanhe periodicamente os pontos com alertas ativos e dispare notificações de aproximação e de perda de rastreamento, sem envio duplicado;

e) tratar explicitamente a falha parcial dos coletores, mantendo a aplicação em funcionamento e informando ao usuário quais informações estão indisponíveis;

f) construir a interface para dispositivos móveis na forma de aplicação web progressiva, instalável e com abertura em modo desconectado;

g) implantar a solução em ambiente de produção acessível publicamente e documentar as decisões de projeto, as limitações e os fundamentos teóricos que as sustentam.

1.2 Motivação

A motivação para o desenvolvimento deste trabalho decorre da experiência cotidiana dos integrantes do grupo como usuários do transporte coletivo da Baixada Santista. O câmpus em que o curso é ministrado situa-se em Santos, e o deslocamento diário dos alunos depende, em grande medida, das linhas municipais e intermunicipais que servem a região.

As operadoras mantêm portais denominados “Quanto Tempo Falta?”, que informam quantos minutos faltam para cada ônibus chegar a determinado ponto. A condição de usuário permitiu identificar três limitações desses portais. A primeira é que eles só respondem quando consultados: o passageiro que deseja saber o momento de sair de casa precisa abrir e atualizar a página repetidas vezes. A segunda é a perda silenciosa de rastreamento: um veículo que estava a caminho simplesmente desaparece da lista, sem qualquer indicação, e o usuário permanece aguardando um ônibus cuja posição deixou de ser conhecida. A terceira é que os portais não foram concebidos para o uso no celular, em deslocamento, com uma única mão e sob luz solar intensa.

Essas limitações orientaram a definição do produto. Em vez de exigir consulta repetida, o sistema inverte o sentido da comunicação: o usuário registra uma única vez o ponto, a linha, o horário habitual e a antecedência desejada, e o servidor passa a vigiar o ponto em seu lugar, enviando a notificação no momento adequado.

Do ponto de vista acadêmico, o tema mostrou-se particularmente adequado à disciplina de Desenvolvimento de Sistemas Distribuídos. A consulta simultânea a fontes independentes, a tolerância a falhas parciais, a comunicação assíncrona do servidor para o cliente e a garantia de que operações repetidas não produzam efeitos duplicados correspondem a problemas de estudo consolidado na área. O trabalho oferece, assim, a oportunidade de aplicar conceitos teóricos da disciplina a um problema real e verificável no cotidiano dos próprios autores.

2 INTRODUÇÃO

O transporte público coletivo constitui elemento estruturante da mobilidade urbana e determina, em grande medida, as condições de acesso da população ao trabalho, à educação e aos serviços públicos. A qualidade percebida desse serviço não depende exclusivamente da oferta de veículos e da frequência de viagens, mas também da previsibilidade que ele oferece ao usuário.

A previsibilidade tem efeito direto sobre o tempo de espera percebido. Um usuário que desconhece o momento de chegada do próximo veículo precisa permanecer no ponto de parada durante todo o intervalo de incerteza, ao passo que aquele que dispõe de informação confiável pode organizar seu deslocamento de modo a reduzir a espera. Essa diferença influencia a percepção de qualidade do serviço e, consequentemente, a disposição da população em utilizá-lo em detrimento do transporte individual motorizado.

Sob a perspectiva ambiental urbana, essa escolha modal possui consequências relevantes. O transporte individual motorizado apresenta consumo energético e emissão de poluentes substancialmente superiores por passageiro transportado, além de demandar maior área viária. Sistemas de informação ao usuário que tornem o transporte coletivo mais previsível atuam, portanto, como instrumento indireto de qualificação ambiental do espaço urbano.

2.1 Contexto regional

A Região Metropolitana da Baixada Santista, instituída pela Lei Complementar Estadual nº 815, de 30 de julho de 1996 (SÃO PAULO, 1996), reúne nove municípios litorâneos articulados em torno do complexo portuário de Santos e concentra população estimada em 1.871.859 habitantes, segundo projeção do Instituto Brasileiro de Geografia e Estatística elaborada a partir do Censo Demográfico de 2022 (IBGE, 2026). A intensa integração funcional entre os municípios manifesta-se em fluxos diários significativos de deslocamento intermunicipal, motivados sobretudo por trabalho e estudo.

O transporte coletivo da região organiza-se em dois níveis. As linhas municipais operam no interior de cada município sob concessão das respectivas prefeituras, cabendo à Viação Piracicabana a operação dos serviços de Santos. As

linhas intermunicipais são geridas pela Empresa Metropolitana de Transportes Urbanos de São Paulo, que administra mais de sessenta linhas metropolitanas nos nove municípios, com transporte diário superior a duzentos mil passageiros, operadas pelo consórcio BR Mobilidade Baixada Santista (EMTU, 2026).

Para o usuário, essa dualidade institucional representa um obstáculo prático. Cada operadora mantém o seu portal, e um mesmo ponto de parada é atendido por linhas dos dois níveis. Um passageiro que deseje saber qual ônibus passará primeiro em determinado ponto precisa consultar dois sistemas separados e comparar mentalmente as respostas.

2.2 Sistemas de informação ao usuário

Os sistemas de informação ao usuário de transporte público evoluíram consideravelmente nas últimas décadas. Inicialmente restritos à divulgação de itinerários e tabelas horárias impressas, passaram a incorporar, com a disseminação do rastreamento por satélite, a divulgação da posição dos veículos e a estimativa do tempo de chegada.

Essa evolução alterou a natureza da informação prestada. A tabela horária constitui informação estática e planejada, que descreve o serviço tal como programado. A previsão de chegada constitui informação dinâmica e observada, que descreve o serviço tal como efetivamente executado. Os portais que atendem a Baixada Santista disponibilizam ambos os tipos de informação, porém segundo o modelo de consulta: a informação existe, mas cabe ao usuário buscá-la no momento certo. Além disso, nenhum tratamento é oferecido para a situação em que um veículo deixa de ser rastreado. É nesse espaço que se insere a contribuição deste trabalho.

2.3 A proposta

Este trabalho propõe uma aplicação distribuída para dispositivos móveis que unifica as informações das duas operadoras da região e transforma a consulta em aviso. O sistema obtém sob demanda, por meio de coletores dedicados a cada operadora, as previsões e os demais dados publicados nos portais; uma central

agrega essas respostas em uma visão regional; e um monitor executado na central acompanha, a cada trinta segundos, os pontos para os quais existam alertas ativos.

Duas situações geram notificação. Na primeira, um veículo da linha escolhida encontra-se a uma distância, em minutos, igual ou inferior à antecedência configurada pelo usuário. Na segunda, um veículo que estava a caminho desaparece da lista de previsões antes de alcançar o ponto, o que indica provável perda de sinal do equipamento de rastreamento. Cada combinação de alerta, veículo, tipo de evento e dia gera no máximo uma notificação.

A aplicação dispensa cadastro e autenticação. O identificador do usuário é a própria inscrição de notificação do dispositivo, gerada pelo navegador, o que reduz o volume de dados pessoais tratados.

2.4 Justificativa da arquitetura distribuída

A adoção de uma arquitetura distribuída decorre de características próprias do problema. As informações provêm de duas fontes independentes, sujeitas a falhas independentes; isolar cada uma delas em um processo próprio impede que a indisponibilidade de uma operadora interrompa o atendimento das demais e permite acrescentar novas fontes sem alterar o restante do sistema. O acompanhamento dos pontos precisa ocorrer de forma contínua e autônoma, independentemente da presença do usuário na aplicação, o que exige um componente servidor permanentemente ativo. A entrega do aviso exige comunicação iniciada pelo servidor em direção a um dispositivo que pode estar com a aplicação fechada, mediada por serviços de notificação mantidos pelos fornecedores dos navegadores. Por fim, a interface precisa ser entregue a dispositivos heterogêneos por meio da rede.

Essas exigências correspondem, respectivamente, a problemas de isolamento de falhas, de execução agendada, de comunicação assíncrona e de transparência de distribuição, tratados no capítulo seguinte.

2.5 Estrutura do documento

O capítulo 3 apresenta os fundamentos conceituais das tecnologias empregadas. O capítulo 4 apresenta o plano de desenvolvimento, com o problema,

o escopo, as fontes de dados, os requisitos, as tecnologias selecionadas, a revisão do plano em relação à versão anterior, a análise de custos, os riscos, o cronograma e a divisão de tarefas. O capítulo 5 detalha o projeto do programa, com a arquitetura, os módulos, o modelo de dados e os diagramas. O capítulo 6 apresenta o relatório com as linhas de código e o capítulo 7, a aplicação em funcionamento. O capítulo 8 discute a interdisciplinaridade e a contribuição do trabalho para a formação dos integrantes, e o capítulo 9 apresenta as conclusões, seguidas do glossário e das referências.

3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS

Este capítulo apresenta os conceitos gerais que fundamentam as tecnologias empregadas no sistema. São abordados os princípios de sistemas distribuídos que sustentam a arquitetura, a detecção e a tolerância a falhas, a idempotência, a concorrência assíncrona, a virtualização por contêineres, a comunicação iniciada pelo servidor, as aplicações web progressivas como plataforma móvel e as técnicas de extração de dados da web.

3.1 Sistemas distribuídos: conceitos gerais

Um sistema distribuído pode ser definido como um conjunto de computadores independentes que se apresenta ao usuário como um sistema único e coerente. Segundo Tanenbaum e Van Steen (2007), essa definição contempla dois aspectos complementares: o primeiro diz respeito ao hardware, uma vez que as máquinas envolvidas são autônomas; o segundo refere-se ao software, pois os usuários percebem o conjunto como uma entidade única.

Coulouris et al. (2013) caracterizam os sistemas distribuídos pela presença de componentes situados em computadores interligados em rede, que se comunicam e coordenam suas ações exclusivamente por meio de troca de mensagens. Dessa definição derivam três consequências fundamentais: a concorrência entre componentes, a ausência de um relógio global e a possibilidade de falhas independentes de cada elemento. A falha independente é a mais relevante para o sistema proposto: a falha de um componente não implica a falha do conjunto, e tampouco é imediatamente perceptível aos demais.

3.1.1 Transparência

A transparência consiste na ocultação, ao usuário, da natureza distribuída do sistema. Tanenbaum e Van Steen (2007) enumeram diversas formas de transparência, entre as quais se destacam, para os fins deste trabalho, a transparência de localização, que oculta o local em que um recurso se encontra, e a transparência de falha, que oculta a ocorrência e a recuperação de falhas. No

sistema proposto, o usuário consulta “os ônibus da região”, sem saber que as respostas provêm de dois coletores e de duas empresas distintas.

Cabe observar que a transparência plena nem sempre é desejável. Em aplicações sensíveis ao tempo, ocultar completamente a ocorrência de falhas leva à exibição de informação incompleta como se estivesse completa. Por essa razão, quando um coletor falha, a aplicação continua operando com os dados disponíveis, mas informa ao usuário qual operadora está fora do ar.

3.1.2 Escalabilidade

A escalabilidade expressa a capacidade de um sistema manter desempenho adequado diante do crescimento do número de usuários, do volume de dados ou da extensão geográfica de sua atuação (COULOURIS et al., 2013). As principais técnicas para obtê-la são a replicação, que mantém múltiplas cópias de um componente; a distribuição, que particiona o trabalho entre componentes distintos; e o armazenamento em cache, que mantém cópias temporárias de dados próximas ao ponto de consumo. Componentes que não guardam estado relevante, denominados sem estado, podem ser replicados e reiniciados livremente, o que torna a separação entre componentes com e sem estado uma decisão central de projeto.

3.1.3 Modelos de comunicação

No modelo de requisição e resposta, também denominado cliente-servidor, um componente solicita um serviço e aguarda a resposta correspondente. Trata-se do modelo predominante na web, materializado no protocolo HTTP (KUROSE; ROSS, 2013). Um cliente que deseja acompanhar continuamente uma informação, nesse modelo, precisa consultá-la repetidamente, prática conhecida como sondagem periódica, cujo custo cresce com o número de clientes ainda que a informação permaneça inalterada.

Quando uma mesma requisição precisa ser respondida por várias fontes, emprega-se o padrão de espalhamento e coleta, conhecido pelo termo em inglês *scatter-gather* (HOHPE; WOOLF, 2003). Um componente agregador envia a requisição a todos os destinatários ao mesmo tempo, aguarda as respostas por um intervalo limitado e as combina em um resultado único. O tempo total da operação

aproxima-se do tempo da fonte mais lenta, e não da soma dos tempos, e a ausência de uma resposta pode ser tratada sem comprometer as demais.

3.2 Detecção e tolerância a falhas

Coulouris et al. (2013) classificam as falhas em sistemas distribuídos conforme a forma pela qual se manifestam. A falha por parada ocorre quando um componente interrompe definitivamente sua operação. A falha por omissão ocorre quando um componente deixa de executar determinada ação esperada, como a entrega de uma mensagem. A falha de temporização ocorre quando a resposta é produzida fora do intervalo temporal esperado.

Em um sistema assíncrono, no qual não há limite superior conhecido para o tempo de transmissão de mensagens, é impossível distinguir com certeza um componente que falhou de um componente que está apenas lento (FISCHER; LYNCH; PATERSON, 1985). Todo mecanismo prático de detecção opera, portanto, com base em suspeita. Chandra e Toueg (1996) formalizaram o conceito de detector de falhas não confiável, caracterizado por duas propriedades em tensão: a completude, que é a capacidade de eventualmente suspeitar de todo componente que efetivamente falhou, e a acurácia, que é a capacidade de não suspeitar de componentes operacionais. O mecanismo mais simples é o tempo limite: decorrido um intervalo sem resposta, o componente é considerado indisponível para aquela operação. Um limite curto detecta falhas rapidamente, mas produz suspeitas indevidas; um limite longo é mais preciso, mas atrasa a reação.

A tolerância a falhas parciais consiste em manter o sistema útil quando parte de seus componentes falha, oferecendo um serviço reduzido em vez de nenhum serviço. Essa degradação controlada exige que o sistema saiba quais respostas faltam e que as decisões tomadas a partir de dados incompletos levem em conta essa incompletude.

3.3 Idempotência e entrega de mensagens

Uma operação é idempotente quando sua execução repetida produz o mesmo efeito que uma execução única. A propriedade é essencial em sistemas distribuídos porque falhas e reinícios levam à repetição de operações: um processo

que reinicia não sabe com certeza quais efeitos já produziu antes de falhar. As garantias de entrega de mensagens costumam ser classificadas em “no máximo uma vez”, “ao menos uma vez” e “exatamente uma vez”; esta última é obtida na prática pela combinação de repetição com deduplicação, em geral apoiada em um identificador único registrado de forma atômica antes do efeito externo (COULOURIS et al., 2013).

Em bancos de dados relacionais, uma restrição de unicidade oferece exatamente esse mecanismo: a tentativa de registrar duas vezes o mesmo identificador é rejeitada pelo próprio gerenciador, de forma atômica, sem necessidade de coordenação adicional entre processos.

3.4 Concorrência assíncrona

Grande parte do trabalho de um servidor web consiste em aguardar operações de entrada e saída, como respostas de rede e leituras de disco. No modelo de programação assíncrona, um laço de eventos executa diversas tarefas de forma cooperativa em uma única linha de execução: quando uma tarefa aguarda uma operação de entrada e saída, cede o controle para que outra prossiga. A linguagem Python oferece esse modelo por meio do módulo asyncio e das palavras reservadas async e await (PYTHON SOFTWARE FOUNDATION, 2026). A função asyncio.gather permite disparar várias requisições ao mesmo tempo e aguardar o conjunto, o que constitui a base técnica do padrão de espalhamento e coleta descrito na seção 3.1.3.

3.5 Virtualização por contêineres

Um contêiner é um processo isolado do sistema operacional hospedeiro por mecanismos do próprio núcleo, com sistema de arquivos, rede e limites de recursos próprios, porém sem a sobrecarga de uma máquina virtual completa. O contêiner é criado a partir de uma imagem, que reúne a aplicação e todas as suas dependências, o que garante que o mesmo artefato execute de forma idêntica no computador do desenvolvedor e no servidor de produção (DOCKER, 2026).

Ferramentas de composição, como o Docker Compose, descrevem em um único arquivo os serviços de uma aplicação, a rede privada pela qual eles se

comunicam e os volumes em que os dados persistem. Cada serviço torna-se um nó lógico do sistema distribuído, endereçável pelo nome na rede interna, e pode ser iniciado, interrompido ou replicado de forma independente. À frente dos serviços, um proxy reverso recebe as requisições externas e as encaminha ao serviço adequado, concentrando a terminação da criptografia TLS.

3.6 Comunicação iniciada pelo servidor: Web Push

O protocolo HTTP foi concebido segundo o modelo em que a comunicação é sempre iniciada pelo cliente. Para entregar uma notificação a um dispositivo cuja aplicação está fechada, os navegadores adotam o padrão Web Push (THOMSON; DAMAGGIO; RAYMOR, 2016). Nele, o navegador registra o dispositivo junto a um serviço de notificação mantido pelo seu fornecedor e recebe um endereço exclusivo, denominado *endpoint*. O servidor da aplicação envia a mensagem a esse endereço, e o serviço de notificação se encarrega de entregá-la ao dispositivo assim que ele estiver acessível.

A autenticidade da origem é garantida pelo esquema VAPID (THOMSON; BECK, 2017): o servidor possui um par de chaves criptográficas, divulga a chave pública ao navegador no momento da inscrição e assina cada envio com a chave privada. O conteúdo da mensagem é cifrado com chaves fornecidas pelo próprio navegador (THOMSON, 2017), de modo que o serviço intermediário não tem acesso ao texto da notificação. Quando o usuário revoga a permissão ou remove a aplicação, o serviço passa a responder ao envio com os códigos 404 ou 410, sinal de que a inscrição deve ser descartada.

3.7 Aplicações web progressivas como plataforma móvel

O desenvolvimento para dispositivos móveis pode seguir três abordagens: a nativa, com linguagem e ferramentas próprias de cada sistema operacional; a multiplataforma, com uma base de código traduzida para cada sistema; e a baseada em tecnologias web, executada no navegador do dispositivo. As aplicações web progressivas constituem um refinamento da terceira abordagem, ao incorporar recursos tradicionalmente associados a aplicativos nativos: instalação na tela inicial, funcionamento sem conexão e recebimento de notificações.

Dois elementos viabilizam esses recursos. O manifesto é um documento JSON que descreve a aplicação ao sistema operacional, informando nome, ícones, cores e modo de exibição, e permite a instalação na tela inicial (W3C, 2026b). O *service worker* é um script executado pelo navegador em segundo plano, independentemente da página, capaz de interceptar requisições de rede para servir a aplicação a partir de cache e de receber as mensagens de Web Push mesmo com a aplicação fechada (W3C, 2026a). Ambos exigem que a aplicação seja servida sob HTTPS. Registra-se uma limitação de plataforma: no sistema iOS, o recebimento de notificações exige que a aplicação tenha sido previamente adicionada à tela inicial, restrição que não se aplica ao Android.

3.8 Extração de dados da web

A extração de dados da web, conhecida pelo termo em inglês *web scraping*, consiste na obtenção automatizada de informações publicadas em páginas destinadas à leitura humana, e é empregada quando o detentor dos dados não disponibiliza interface programática. O cenário mais favorável ocorre quando as páginas são geradas no servidor: basta reproduzir a requisição que o navegador faria e analisar o documento HTML retornado, localizando os elementos de interesse por sua estrutura. Bibliotecas como a Beautiful Soup oferecem essa análise em Python (RICHARDSON, 2026).

A principal limitação da técnica é sua dependência da estrutura da página de origem, que pode ser alterada a qualquer momento e sem aviso. A mitigação recomendada consiste em isolar a lógica de extração em componente específico, submetê-la a testes automatizados executados sobre cópias reais das páginas e prever comportamento degradado, porém seguro, diante de falhas de interpretação.

A extração suscita ainda considerações éticas e legais. No plano da conduta, cabe limitar a frequência de acesso para não impor carga desproporcional à infraestrutura de origem. No plano da proteção de dados, a Lei nº 13.709/2018 (BRASIL, 2018) estabelece que o tratamento de dados relativos a pessoa natural requer fundamento em hipótese legal, e a circunstância de um dado estar publicamente acessível não autoriza, por si só, seu armazenamento ou sua redistribuição. Dessa constatação decorre a orientação, adotada neste projeto, de não persistir informações pessoais presentes na fonte.

4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

4.1 Descrição do problema

Os portais “Quanto Tempo Falta?” da Viação Piracicabana e da BR Mobilidade Baixada Santista informam, para cada ponto de parada, os veículos que se aproximam e a previsão de chegada em minutos. O problema abordado decorre de três características desses portais, descritas na seção 1.2: dependem de consulta ativa e repetida, removem sem advertência os veículos que perdem o rastreamento e não são adequados ao uso em dispositivos móveis. Soma-se a elas a fragmentação: os códigos dos pontos são compartilhados entre os dois portais, de modo que o mesmo ponto 9306, por exemplo, é atendido por linhas municipais em um portal e por linhas intermunicipais em outro.

4.2 Escopo

O sistema abrange os dois portais que atendem a região, unificando em uma única interface as linhas municipais da Piracicabana e as linhas intermunicipais da BR Mobilidade Baixada Santista.

Integram o escopo a busca de pontos e linhas, a identificação de pontos próximos ao usuário, a exibição das previsões de chegada de um ponto com atualização automática, a consulta do itinerário, do traçado, da tabela horária e da posição ao vivo dos veículos de uma linha, a criação, o teste e a exclusão de alertas, o envio de notificações de aproximação e de perda de rastreamento e o tratamento de falhas parciais dos coletores.

Não integram o escopo a recarga de cartões de transporte, a venda de passagens, a roteirização ponto a ponto, o cadastro de usuários, o cálculo próprio de estimativa de chegada, que é obtida da própria operadora, e a produção de indicadores históricos de pontualidade, previstos na revisão anterior e deslocados para trabalhos futuros.

4.3 Fontes de dados

Os portais das duas operadoras executam o mesmo sistema, diferindo apenas na identidade visual e no endereço. A estrutura das requisições e o formato das respostas são idênticos, o que permite um único analisador, parametrizado pelo endereço da operadora. As páginas são geradas no servidor e não exigem autenticação, o que dispensa navegador automatizado.

Quadro 1 – Recursos dos portais utilizados na coleta

Recurso	Requisição	Conteúdo obtido
select_option.php	POST var_pesq	Pontos cujo endereço contém o texto buscado (mínimo de três letras)
atualiza_previsao.php	POST idPonto	Previsões até 20 min (linha, prefixo do veículo, primeiro nome do condutor) e linhas com previsão acima de 20 min
update_bus.php	POST linha_id	Posições ao vivo dos veículos da linha, com sentido e horário da posição
pg_FindLines.php	GET	Catálogo de linhas da operadora
pg_FindLinesDt.php	GET idLinha	Descrição, itinerário por logradouro e tabela horária programada
pg_mapaLinha.php	GET idLinha	Traçado de ida e volta, pontos com coordenadas e número interno da linha

Fonte: Autor (2026).

Algumas particularidades da fonte condicionaram a implementação. Sem um cabeçalho de identificação de navegador, os servidores respondem com o código 403. Os servidores não enviam o certificado intermediário da cadeia TLS; em vez de desligar a verificação, o projeto acrescenta esse certificado ao conjunto de autoridades confiáveis. O rótulo de uma linha varia entre páginas (“04” em uma, “CIRCULAR 004” em outra), razão pela qual o identificador estável adotado é o código de quarenta caracteres hexadecimais idLinha. A previsão “CHEGANDO!” corresponde a três minutos ou menos e é tratada como zero. Por fim, previsões com veículo identificado só são fornecidas até vinte minutos; acima disso, o portal informa apenas as linhas que passam pelo ponto.

4.3.1 Conduta de coleta

A coleta ocorre sob demanda: os coletores só consultam os portais quando a aplicação ou o monitor solicitam uma informação. O detalhe de cada linha, que quase nunca muda, permanece seis horas em cache na central, ao passo que previsões e posições nunca passam por cache. O catálogo de pontos com

coordenadas, necessário ao mapa de pontos próximos, é reconstruído uma vez por dia, com no máximo três consultas simultâneas e pausas entre elas. Os dados são de titularidade das operadoras e da EMTU, e sua utilização restringe-se à finalidade acadêmica.

4.4 Requisitos funcionais

Quadro 2 – Requisitos funcionais

ID	Descrição
RF01	Buscar pontos por endereço ou código e linhas por número ou denominação
RF02	Exibir os pontos próximos ao usuário em mapa, a partir da geolocalização do dispositivo
RF03	Exibir as previsões de chegada de um ponto, reunindo as duas operadoras, ordenadas pela chegada
RF04	Atualizar as previsões automaticamente, a cada 15 segundos, sem ação do usuário
RF05	Exibir itinerário, traçado, pontos e tabela horária de uma linha
RF06	Exibir a posição ao vivo dos veículos de uma linha no mapa
RF07	Criar alerta com ponto, linha, horário habitual e antecedência
RF08	Listar, testar e excluir os alertas do dispositivo
RF09	Notificar a aproximação do veículo quando a previsão for igual ou inferior à antecedência
RF10	Notificar o desaparecimento de veículo que estava a caminho
RF11	Não enviar a mesma notificação mais de uma vez por alerta, veículo, tipo e dia
RF12	Informar ao usuário quando uma operadora estiver indisponível
RF13	Operar sem cadastro, utilizando a inscrição de notificação como identificador
RF14	Registrar localmente os pontos consultados recentemente
RF15	Permitir a instalação na tela inicial e a abertura sem conexão

Fonte: Autor (2026).

4.5 Requisitos não funcionais

Quadro 3 – Requisitos não funcionais

ID	Descrição
RNF01	Componentes executados como processos independentes, um por contêiner
RNF02	Comunicação entre nós exclusivamente por HTTP com conteúdo JSON
RNF03	Um coletor por operadora, com a mesma imagem e configuração por variável de ambiente
RNF04	Tempo limite de 10 segundos por coletor em cada requisição da central
RNF05	Falha de um coletor não interrompe a aplicação
RNF06	Estado persistente concentrado na central; coletores reiniciáveis sem perda
RNF07	Ciclo do monitor idempotente
RNF08	Comunicação externa exclusivamente sobre HTTPS, com verificação de certificado
RNF09	Aplicação e interface de programação na mesma origem
RNF10	Não persistência da localização do usuário nem de dados pessoais da fonte
RNF11	Interface legível ao sol, operável com uma mão, com alvos de toque de no mínimo 44 px
RNF12	Respeito à preferência de movimento reduzido do sistema
RNF13	Analisador coberto por testes automatizados sobre páginas reais
RNF14	Serviços executados sem privilégios de administrador

Fonte: Autor (2026).

4.6 Tecnologias selecionadas

A seleção privilegiou ferramentas maduras, de código aberto e simples de explicar, adequadas a um trabalho acadêmico desenvolvido por equipe com afinidades técnicas distintas.

Quadro 4 – Tecnologias selecionadas

Camada	**Tecnologia**	**Justificativa**
Linguagem do servidor	Python 3.12	Ecossistema amplo para extração de dados e sintaxe acessível a todo o grupo
Interface de programação	FastAPI e Uvicorn	Servidor assíncrono, com validação automática dos dados e documentação gerada
Validação de dados	Pydantic	Modelos tipados para as mensagens trocadas entre os nós
Cliente HTTP	httpx	Requisições assíncronas, base do espalhamento e coleta
Extração de dados	Beautiful Soup	Análise tolerante de documentos HTML
Notificações	pywebpush	Envio de Web Push com assinatura VAPID
Persistência	SQLite	Banco relacional em arquivo, sem servidor adicional
Testes	pytest	Testes sobre cópias reais das páginas das operadoras
Interface	SvelteKit, Svelte 5 e TypeScript	Compilação antecipada, menor volume de código entregue ao dispositivo
Estilo e mapas	Tailwind CSS e Leaflet	Estilização consistente e mapas sobre OpenStreetMap
Servidor web	Caddy	Entrega da aplicação e encaminhamento de /api à central
Contêineres	Docker e Docker Compose	Execução idêntica dos quatro nós em qualquer máquina
Implantação	Dokploy e Traefik	Publicação a partir do repositório, com certificado TLS automático

Fonte: Autor (2026).

4.6.1 Aplicação web progressiva como plataforma móvel

A aplicação é desenvolvida como aplicação web progressiva, instalável na tela inicial e com recebimento de notificações. A opção fundamenta-se na manutenção de base de código única para Android e iOS, na dispensa de distribuição por lojas de aplicativos e no fato de o produto depender essencialmente de notificações, recurso disponível na plataforma web. A demonstração é realizada em dispositivo Android.

4.7 Revisão do plano em relação à Revisão 02

A Revisão 02 previa a execução em nós de borda de uma plataforma de nuvem, com componentes de estado por linha, cálculo próprio da estimativa de chegada a partir da posição dos veículos, detectores de falha por acúmulo de suspeita e indicadores históricos. Durante a investigação das fontes e o início da implementação, três constatações motivaram a reorientação.

A primeira foi a descoberta de que os portais fornecem, por ponto, a previsão de chegada já calculada pela operadora. O algoritmo próprio de projeção sobre o traçado, núcleo técnico da revisão anterior, deixou de ser necessário para entregar o valor principal ao usuário, e a complexidade correspondente foi removida. A segunda foi o custo de complexidade da plataforma anterior: seus limites gratuitos restringiam o monitoramento a quinze ou vinte linhas e exigiam agregação de escritas, e o modelo de componentes de estado tornava a arquitetura difícil de reproduzir localmente e de explicar. A terceira foi a adequação do ecossistema Python à extração de dados e a familiaridade do grupo com a linguagem.

Quadro 5 – Comparação entre as revisões do plano

Aspecto	**Revisão 02**	**Revisão 03**
Execução	Funções em nós de borda de plataforma de nuvem	Serviços em contêineres Docker, em servidor próprio
Linguagem do servidor	TypeScript	Python
Estado	Um componente de estado por linha	Central única com SQLite; coletores sem estado relevante
Estimativa de chegada	Algoritmo próprio sobre o traçado	Previsão fornecida pela operadora
Comunicação com o cliente	Conexão persistente por WebSocket	Consulta a cada 15 s na tela e Web Push para os avisos
Unidade de monitoramento	Linha	Ponto com alerta ativo
Detecção de falhas	Detector por acúmulo e saúde da fonte	Tempo limite por coletor e regra de desaparecimento
Indicadores históricos	Previstos	Trabalhos futuros

Fonte: Autor (2026).

A mudança preservou os objetivos de produto e os fundamentos de sistemas distribuídos: o sistema continua composto por nós independentes que se comunicam por mensagens, continua tratando falhas parciais e continua entregando informação ao dispositivo por iniciativa do servidor. O que se alterou foi a distribuição das responsabilidades, agora organizada por fonte de dados, e não por linha.

4.8 Análise de custos e de escalabilidade

A solução é implantada em um servidor virtual privado já disponível a um dos integrantes, gerenciado pela plataforma de código aberto Dokploy, sem custo adicional de licenças. Todas as bibliotecas empregadas são de código aberto, e o envio de notificações pelos serviços dos navegadores é gratuito. O custo marginal do projeto restringe-se, portanto, ao domínio utilizado na publicação.

A carga imposta às fontes é dominada pelo monitor. Cada alerta permanece ativo durante uma janela de sessenta minutos, de trinta minutos antes a trinta minutos depois do horário configurado; com ciclos de trinta segundos, isso corresponde a cento e vinte ciclos por dia. Em cada ciclo, a central consulta uma vez cada ponto com alerta ativo, independentemente de quantos alertas existam nesse ponto, e cada consulta gera uma requisição a cada um dos dois coletores. Um ponto com alertas ativos produz, portanto, cerca de duzentas e quarenta requisições diárias aos portais, quantidade inferior à gerada por um único usuário que mantenha o portal oficial aberto, já que este se atualiza a cada quinze segundos.

Quanto ao crescimento, os coletores não guardam estado relevante e podem ser replicados, e uma nova operadora corresponde apenas a um novo coletor configurado com outro endereço. A central é o componente com estado e, por isso, o limite de escala e o ponto único de falha do sistema. Para o volume previsto em um trabalho acadêmico, o banco em arquivo é suficiente; a migração para um gerenciador de banco de dados em servidor e a replicação da central constituem o caminho de evolução, registrado entre os trabalhos futuros.

4.9 Riscos identificados

Quadro 6 – Riscos e mitigações

Risco	Severidade	Mitigação
Alteração da estrutura dos portais	Alta	Analisador isolado e testado sobre cópias reais das páginas; falha de interpretação tratada como indisponibilidade do coletor
Bloqueio de acesso automatizado pelos portais	Média	Coleta sob demanda, consultas limitadas e cache dos dados estáticos
Indisponibilidade de uma operadora	Média	Resposta parcial com aviso ao usuário; monitor não infere desaparecimento a partir de dados parciais
Falso aviso de desaparecimento	Média	Veículo a três minutos ou menos não é considerado; comparação apenas entre quadros completos

Risco	Severidade	Mitigação
Perda da chave VAPID	Alta	Chave mantida em volume persistente incluído no backup
Limitação de notificações no iOS	Baixa	Orientação de instalação na tela inicial exibida pela aplicação
Tratamento indevido de dado pessoal de condutor	Alta	Primeiro nome apenas repassado como publicado, sem persistência

Fonte: Autor (2026).

4.10 Cronograma

O cronograma considera disponibilidade individual de oito horas semanais e foi revisado em razão da mudança de plataforma. As etapas concluídas até a data desta revisão estão indicadas no quadro.

Quadro 7 – Cronograma de desenvolvimento

Semana	Atividade	Situação
1	Investigação das fontes de dados e revisão do plano	Concluída
2	Modelos de dados e analisador dos portais, com testes	Concluída
3	Coletor, central e agregação regional	Concluída
4	Persistência dos alertas, monitor e envio de Web Push	Concluída
5	Interface: busca, painel do ponto e alertas; service worker	Concluída
6	Catálogo de pontos, mapa de pontos próximos e página da linha; implantação	Concluída
7	Testes em campo com dispositivos Android e iOS	Em andamento
8	Ajustes de interface e de regras de notificação	Prevista
9	Revisão final da documentação	Prevista
10	Preparação da apresentação	Prevista

Fonte: Autor (2026).

4.11 Divisão de tarefas

A divisão considera a afinidade técnica de cada integrante e mantém interfaces bem definidas entre os módulos, de modo a permitir desenvolvimento paralelo.

Quadro 8 – Divisão de tarefas

Integrante	Frente de trabalho
Enzo Santos Pita	Arquitetura geral, central, monitor, notificações, contêineres, implantação e integração
Daniele dos Santos França	Extração de dados dos portais e testes do analisador
Christyan Charlles	Catálogo de pontos, coordenadas e mapa de pontos próximos
Welber Willian da Silva	Interface da aplicação e experiência do usuário
Luiz Santana de Araujo	Interface da aplicação, aplicação web progressiva e service worker

Fonte: Autor (2026).

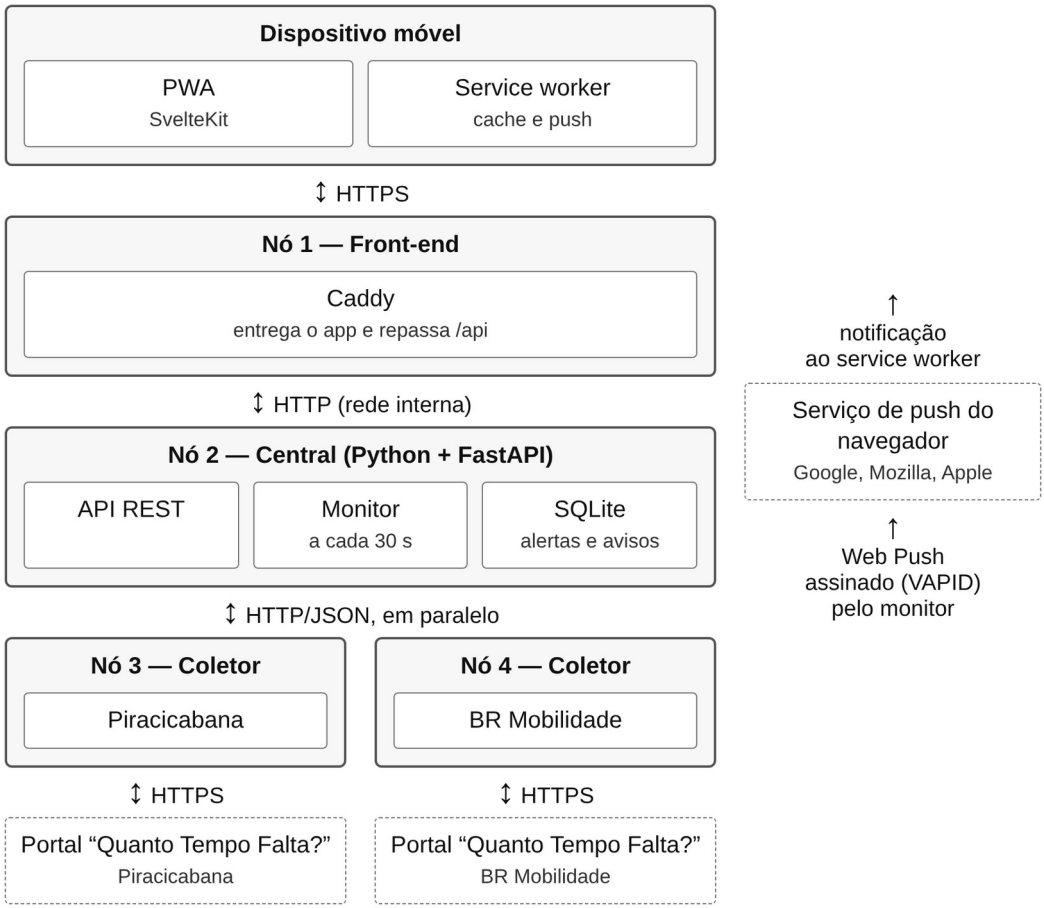
A documentação do trabalho e a preparação da apresentação constituem responsabilidade compartilhada por todos os integrantes.

5 PROJETO DO PROGRAMA

5.1 Arquitetura geral

O sistema é composto por quatro nós servidores e pela aplicação executada no dispositivo do usuário, conforme a Figura 1. O front-end entrega a aplicação e encaminha as chamadas à interface de programação; a central concentra o estado, agrega as respostas e executa o monitor; e cada coletor atende uma operadora, convertendo as páginas do portal em mensagens JSON. A comunicação entre os nós é feita por HTTP síncrono, e o aviso chega ao dispositivo por meio do serviço de notificação do navegador.

Figura 1 – Arquitetura geral do sistema



Fonte: Autor (2026).

Quadro 9 – Responsabilidades dos nós

Nó	Responsabilidade	Estado
Aplicação (PWA)	Telas, inscrição de notificação e exibição do aviso	Cache da aplicação e pontos recentes, no dispositivo

Nó	Responsabilidade	Estado
Front-end (Caddy)	Entrega dos arquivos e encaminhamento de /api à central, na mesma origem	Nenhum
Central	Agregação regional, alertas, monitor e envio das notificações	Banco SQLite e chave VAPID
Coletor	Extração dos dados de uma operadora, sob demanda	Catálogo de pontos, reconstruível

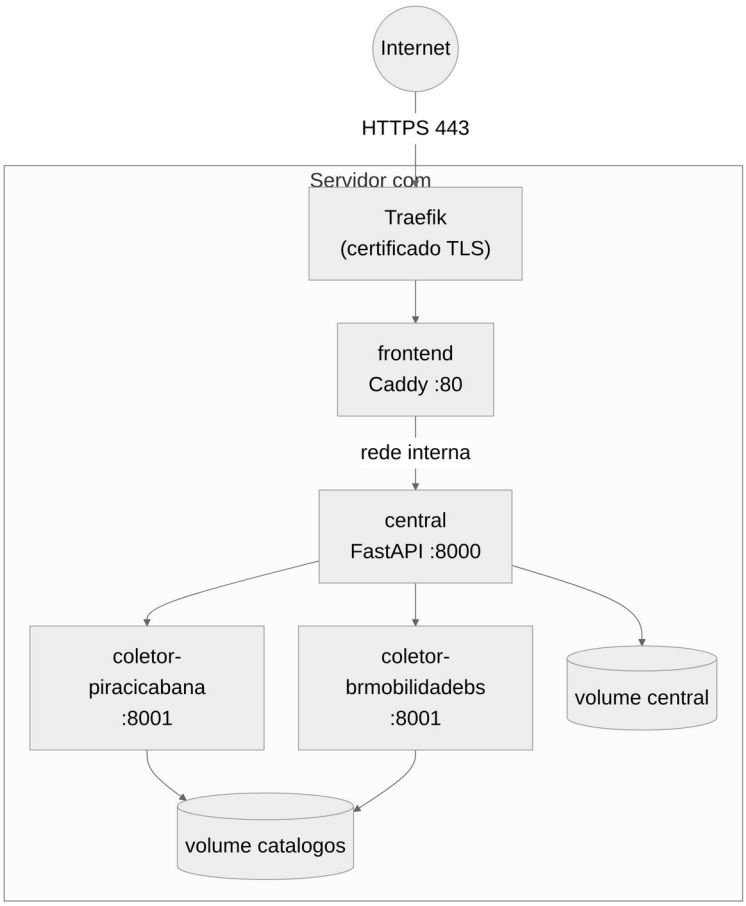
Fonte: Autor (2026).

A colocação da aplicação e da interface de programação na mesma origem é deliberada: o service worker e a inscrição de notificação ficam vinculados à origem da página, e a mesma origem dispensa a configuração de compartilhamento de recursos entre origens. O navegador fala apenas com o front-end, que repassa internamente as chamadas à central, a qual não é exposta à internet.

5.2 Implantação

Os quatro nós são descritos em um arquivo do Docker Compose, conforme a Figura 2, e publicados pela plataforma Dokploy, que constrói as imagens a partir do repositório. O roteador Traefik recebe as conexões externas, emite o certificado TLS, requisito do Web Push, e as encaminha ao front-end. Os dois coletores utilizam a mesma imagem da central, diferindo apenas no comando de inicialização e na variável que indica o endereço da operadora. Os dados persistem em dois volumes: o da central, com o banco e a chave VAPID, e o dos catálogos de pontos. As imagens instalam as dependências antes de copiar o código, o que permite reaproveitá-las entre implantações, e os serviços Python executam sem privilégios de administrador.

Figura 2 – Implantação em produção



Fonte: Autor (2026).

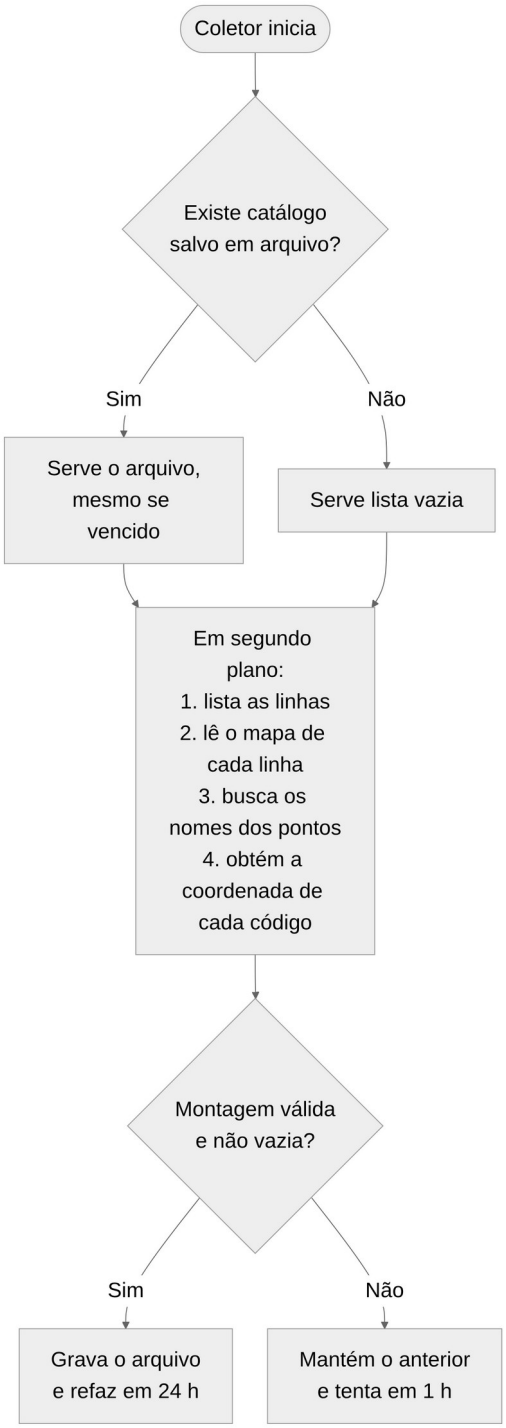
5.3 Módulo coletor

O coletor é um serviço FastAPI que expõe, em formato JSON, as informações de uma operadora: busca de pontos, quadro de previsões de um ponto, catálogo de pontos com coordenadas, lista de linhas, detalhe de uma linha e posições dos veículos. Cada requisição recebida é traduzida em uma requisição ao portal, com o cabeçalho de navegador exigido, e a resposta HTML é convertida pelo analisador em objetos validados pelos modelos Pydantic. O coletor guarda em memória o número interno de cada linha já vista, de modo que a leitura das posições custa uma única requisição ao portal.

O portal não oferece busca por proximidade. Para viabilizar o mapa de pontos próximos, cada coletor monta em segundo plano um catálogo com o código, o nome e a coordenada de todos os pontos da operadora, segundo o procedimento da Figura 3. O catálogo é gravado em arquivo e substituído apenas por uma montagem

bem-sucedida e não vazia, de modo que uma falha transitória do portal nunca apaga um catálogo válido.

Figura 3 – Montagem do catálogo de pontos pelo coletor

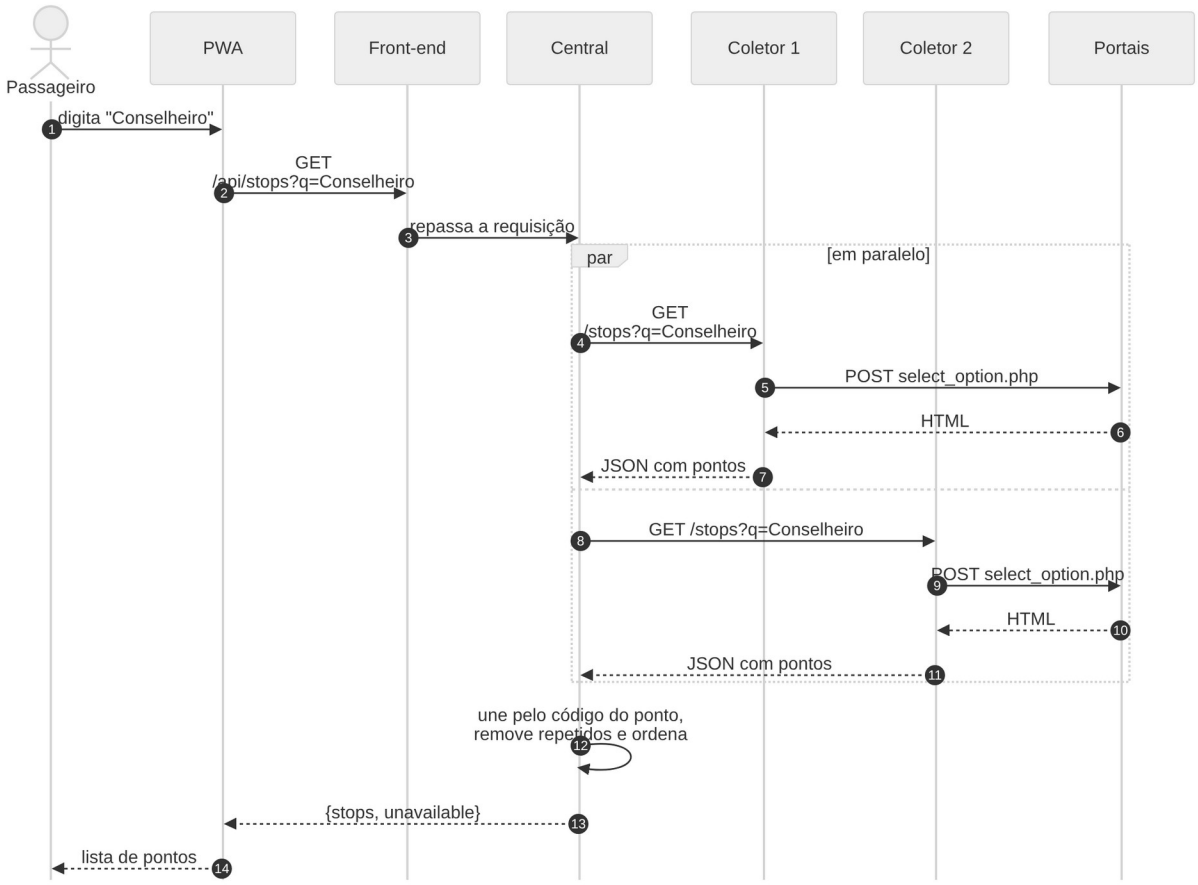


Fonte: Autor (2026).

5.4 Central e agregação regional

A central conhece os coletores por meio de uma variável de ambiente que associa o nome de cada operadora ao seu endereço. Toda consulta regional segue o padrão de espalhamento e coleta: a central envia a requisição a todos os coletores ao mesmo tempo, com tempo limite de dez segundos para cada um, e combina as respostas recebidas. A Figura 4 ilustra a busca de pontos, em que o coletor 1 atende a Piracicabana e o coletor 2, a BR Mobilidade.

Figura 4 – Busca de pontos com espalhamento e coleta



Fonte: Autor (2026).

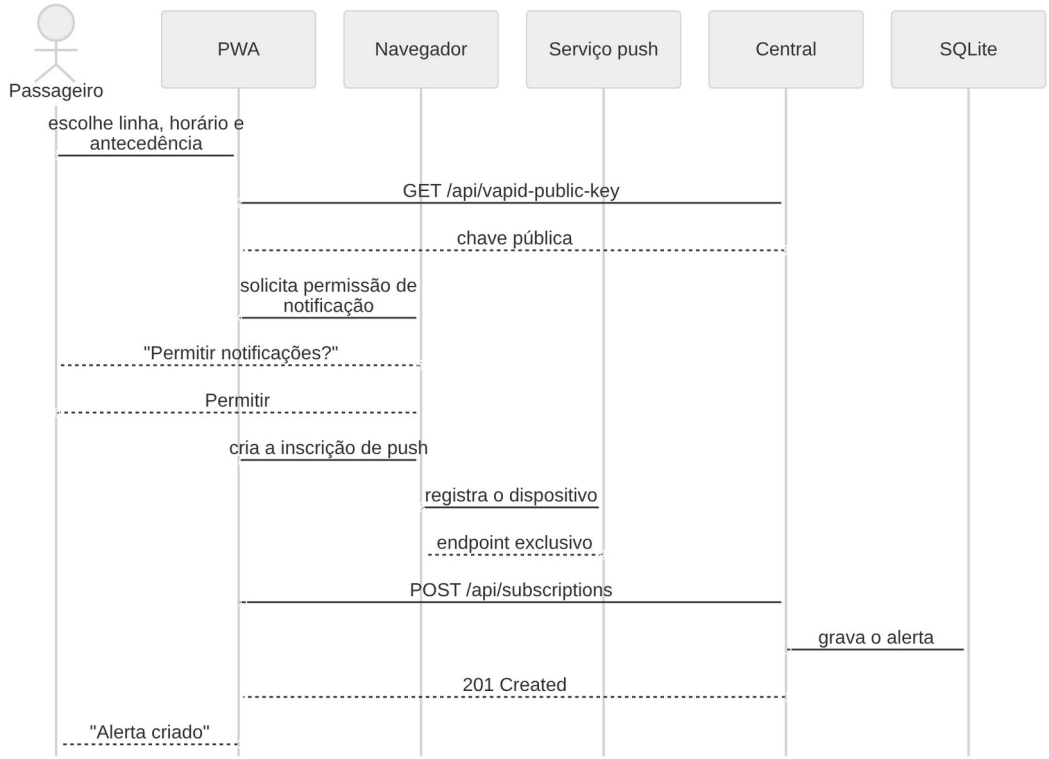
As regras de combinação dependem do recurso. Na busca, os pontos são unidos pelo código, mantendo-se o primeiro nome encontrado, e ordenados alfabeticamente. No quadro de previsões, os veículos das duas operadoras são reunidos e ordenados pela previsão, e cada um recebe a identificação da operadora, utilizada pela interface para colorir a linha conforme seu tipo. Na consulta de uma linha, a central pergunta a todos os coletores e adota o primeiro que a conhece; se nenhum a conhece e algum falhou, responde com o código 502, porque a linha pode

pertencer justamente à operadora indisponível, e só responde 404 quando todos responderam negativamente. O detalhe de uma linha permanece seis horas em cache na memória da central.

5.5 Criação de alertas

O alerta associa um ponto, uma linha, um horário aproximado e uma antecedência em minutos à inscrição de notificação do dispositivo. A Figura 5 apresenta a sequência de criação. A aplicação obtém a chave pública VAPID da central, solicita ao usuário a permissão de notificação, cria a inscrição junto ao serviço do navegador e envia à central os dados do alerta acompanhados do *endpoint* e das chaves de cifragem. A combinação de *endpoint*, ponto, linha e horário é única: registrar novamente o mesmo alerta apenas atualiza a antecedência.

Figura 5 – Sequência de criação de um alerta

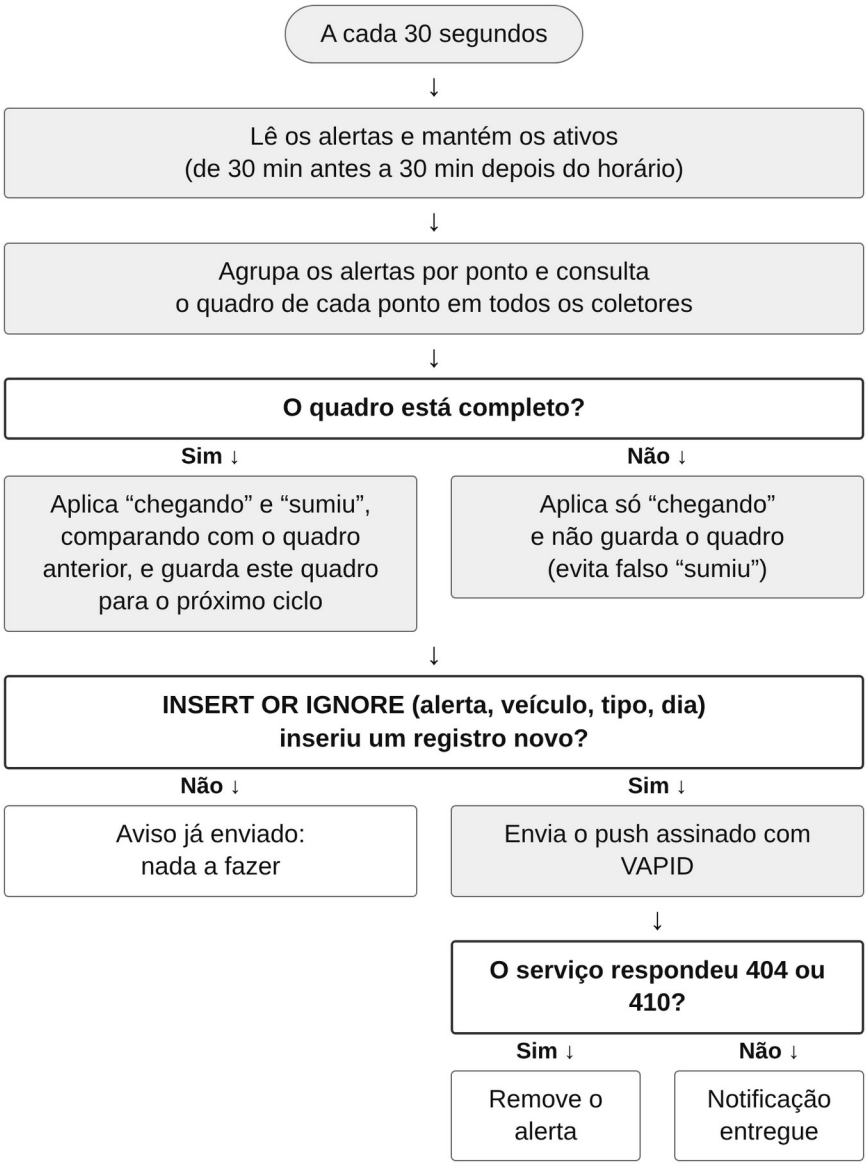


Fonte: Autor (2026).

5.6 Monitor e regras de notificação

O monitor é uma tarefa assíncrona iniciada junto com a central, que executa um ciclo a cada trinta segundos, conforme a Figura 6. Em cada ciclo, seleciona os alertas ativos, isto é, aqueles cujo horário dista no máximo trinta minutos do instante corrente no fuso de São Paulo, considerando a virada do dia, e os agrupa por ponto, de modo que cada ponto seja consultado uma única vez, independentemente do número de alertas.

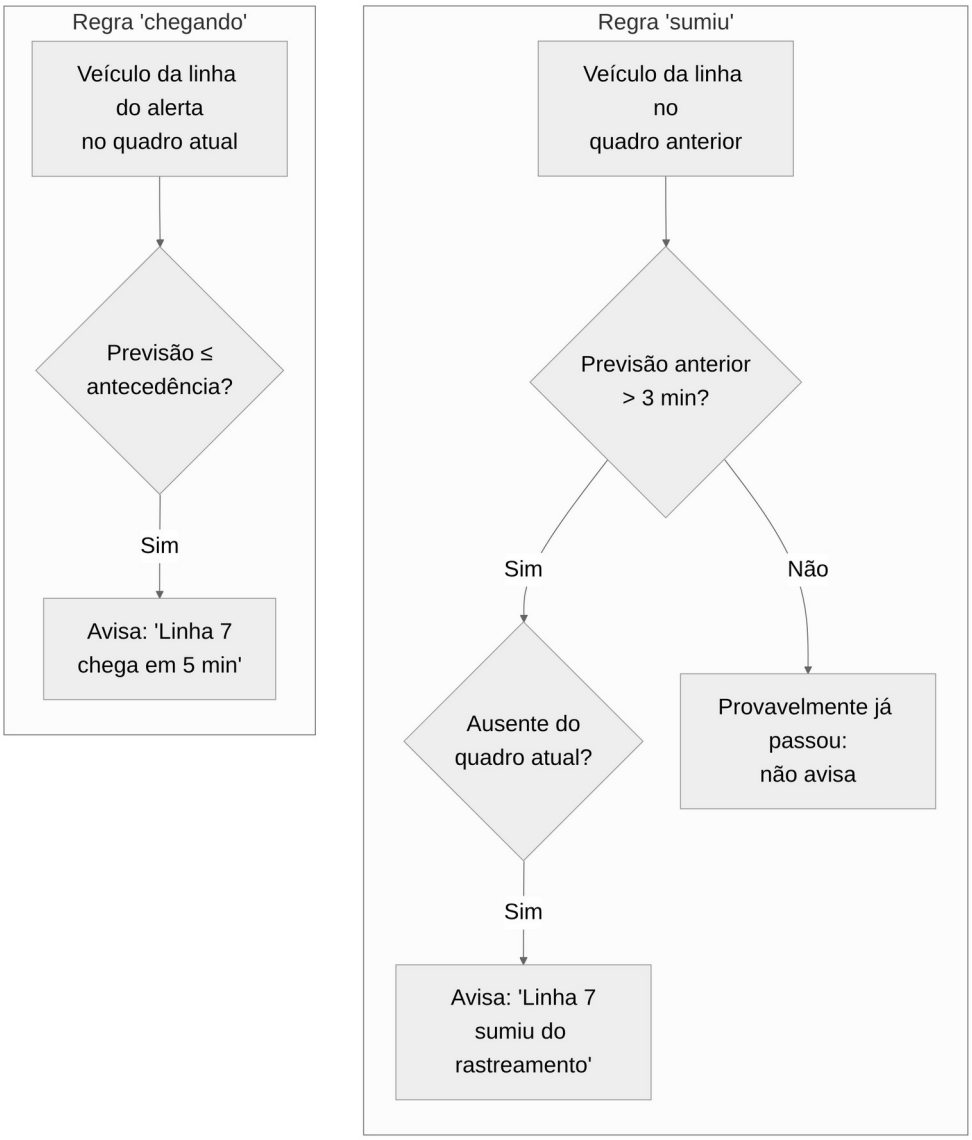
Figura 6 – Ciclo do monitor



Fonte: Autor (2026).

Para cada ponto, o quadro atual é comparado com o quadro do ciclo anterior, segundo as duas regras da Figura 7. A regra de aproximação dispara quando um veículo da linha do alerta apresenta previsão igual ou inferior à antecedência. A regra de desaparecimento dispara quando um veículo da linha, presente no quadro anterior com previsão superior a três minutos, não consta do quadro atual. O limiar de três minutos evita falsos avisos: um veículo com previsão “CHEGANDO!” que deixa a lista provavelmente acabou de passar pelo ponto.

Figura 7 – Regras de notificação



Fonte: Autor (2026).

O Quadro 10 ilustra o comportamento para um alerta no ponto 9306, linha 7, às 7h30, com antecedência de cinco minutos.

Quadro 10 – Exemplo de funcionamento do monitor

Horário	Situação no portal	Ação do sistema
06:50	Veículo 1234 a 12 min	Nenhuma: o alerta só fica ativo às 07:00
07:10	Veículo 1234 a 12 min	Alerta ativo; 12 > 5; o quadro é guardado
07:10:30	Veículo 1234 a 8 min	Nenhuma; o quadro é guardado
07:11	Veículo 1234 a 5 min	Notificação: “Linha 7 a caminho: chega em 5 min”
07:11:30	Veículo 1234 a 4 min	Nenhuma: o aviso deste veículo já foi enviado hoje

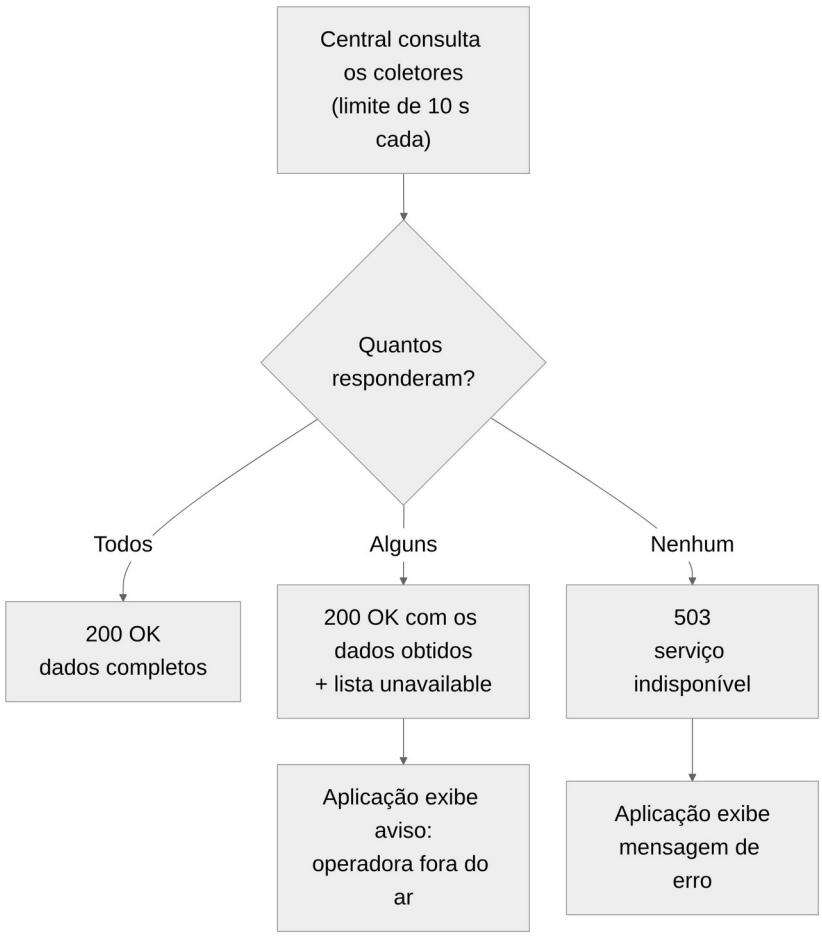
Fonte: Autor (2026).

A garantia de envio único apoia-se na idempotência descrita na seção 3.3. A tabela de notificações possui restrição de unicidade sobre alerta, veículo, tipo e dia, e o monitor tenta registrar a notificação antes de enviá-la: apenas quando o registro é efetivamente inserido o envio ocorre. Repetir um ciclo, ou reiniciar a central no meio de um, não produz avisos duplicados. Quando o serviço de notificação responde 404 ou 410, a inscrição deixou de existir e o alerta é removido.

5.7 Tratamento de falhas parciais

Os coletores dependem de portais de terceiros, que podem ficar indisponíveis. O sistema foi projetado para degradar de forma controlada, segundo a Figura 8. Quando parte dos coletores falha, a central responde com o que obteve e acrescenta a lista das operadoras indisponíveis, que a interface apresenta como aviso discreto; somente quando todos falham a resposta é o código 503.

Figura 8 – Resposta da central à falha de coletores



Fonte: Autor (2026).

O monitor exige cuidado adicional. Quando um coletor falha, os veículos daquela operadora desaparecem do quadro, o que dispararia avisos falsos de desaparecimento. Por isso, um quadro parcial só é avaliado pela regra de aproximação e nunca é guardado como quadro anterior; somente quadros completos servem de referência para o ciclo seguinte. O estado “anterior” é reconstruído a cada ciclo apenas com os quadros lidos por inteiro, de modo que nenhum quadro velho é comparado com um quadro atual.

Quadro 11 – Comportamento diante de falhas

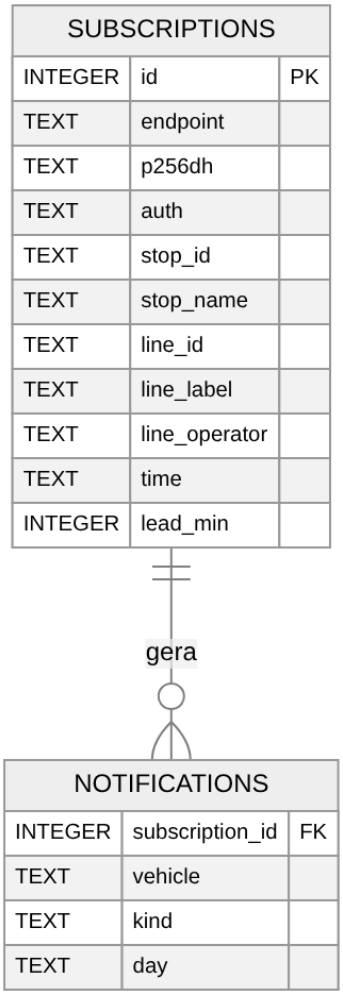
Situação	**O que o usuário vê**
Os dois coletores operando	Informação completa
Um coletor fora do ar	Dados da outra operadora e aviso de qual operadora está indisponível
Os dois coletores fora do ar	Mensagem de erro
Dispositivo sem conexão	A aplicação abre a partir do cache, sem previsões

Fonte: Autor (2026).

5.8 Modelo de dados

O estado persistente restringe-se a duas tabelas na central, apresentadas na Figura 9. Pontos, linhas e horários não são armazenados em banco: são consultados nos portais sob demanda e, quando estáticos, mantidos em cache.

Figura 9 – Modelo de dados da central



Fonte: Autor (2026).

Quadro 12 – Entidades do modelo de dados

Entidade	**Conteúdo principal**
subscriptions	Alerta: *endpoint* e chaves da inscrição de notificação, ponto, linha, operadora, horário e antecedência; único por *endpoint*, ponto, linha e horário
notifications	Notificação enviada: alerta, veículo, tipo (aproximação ou desaparecimento) e dia; única pela combinação dos quatro campos, com exclusão em cascata

Fonte: Autor (2026).

5.9 Interface de programação

A central expõe uma interface REST sob o prefixo /api, consumida pela aplicação por meio de um cliente tipado em TypeScript.

Quadro 13 – Rotas da interface de programação da central

Rota	Função
GET /api/stops?q=	Busca pontos em todos os coletores
GET /api/stops/catalog	Pontos da região com coordenadas
GET /api/stops/{id}/board	Previsões unidas de todas as operadoras
GET /api/lines?q=	Busca linhas por nome ou número
GET /api/lines/{id}	Itinerário, traçado, pontos e horários da linha
GET /api/lines/{id}/vehicles	Posições ao vivo dos veículos, sem cache
GET /api/status	Disponibilidade de cada coletor
GET /api/vapid-public-key	Chave pública VAPID
POST /api/subscriptions	Cria um alerta
GET /api/subscriptions?endpoint=	Lista os alertas do dispositivo
DELETE /api/subscriptions/{id}	Exclui um alerta
POST /api/subscriptions/{id}/test	Envia notificação de teste

Fonte: Autor (2026).

5.10 Interface do usuário

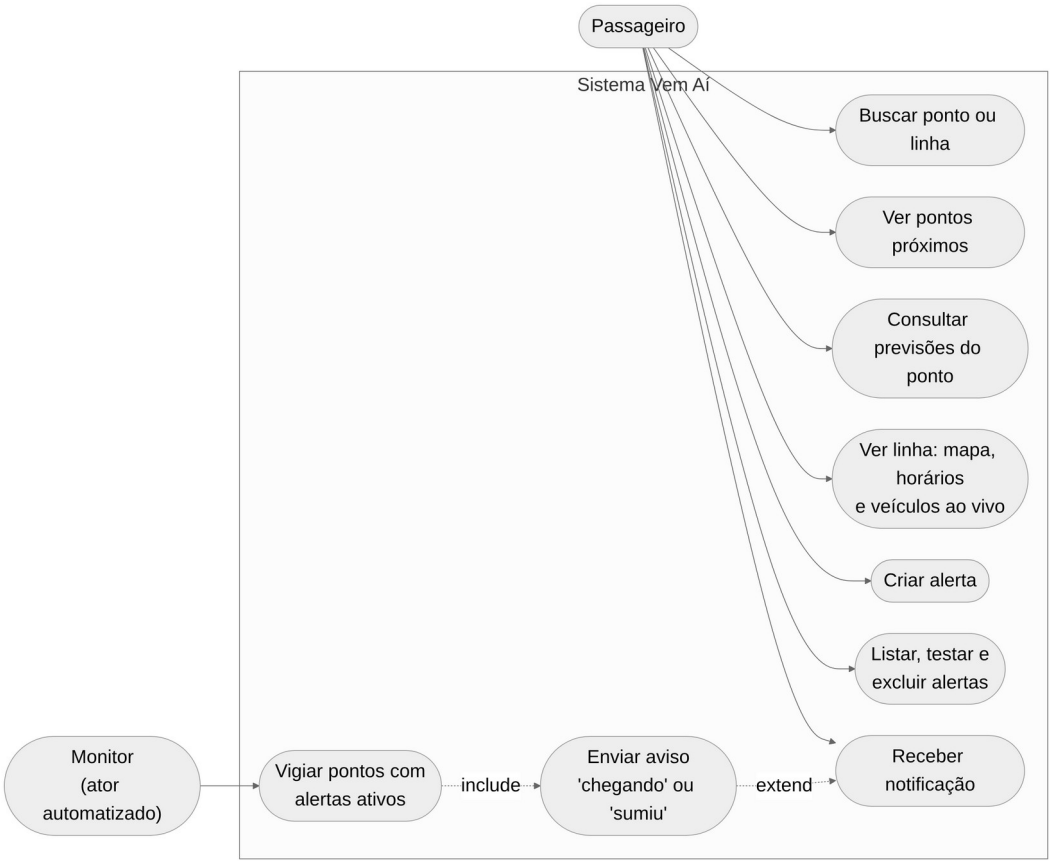
A interface compreende quatro telas, concebidas para uso na rua, com hierarquia visual simples, contraste elevado e alvos de toque amplos. A tela inicial reúne a busca, o mapa de pontos próximos e os pontos recentes. A tela do ponto apresenta as previsões das duas operadoras, atualizadas a cada quinze segundos, com a cor da linha indicando o tipo, verde para municipal e azul para intermunicipal, e o acesso à criação do alerta. A tela da linha apresenta o traçado, os pontos, os veículos ao vivo e a tabela horária. A tela de alertas lista os avisos do dispositivo, com opções de teste e exclusão. O toque na notificação abre diretamente a tela do ponto correspondente.

A aplicação é compilada como aplicação de página única estática. O service worker armazena em cache os arquivos da aplicação para abertura sem conexão, recebe as mensagens de Web Push e trata o toque na notificação. As preferências e os pontos recentes permanecem no próprio dispositivo, e a posição do usuário, utilizada para os pontos próximos, nunca é enviada ao servidor: o cálculo da distância é feito no dispositivo, sobre o catálogo de pontos.

5.11 Diagrama de casos de uso

Os atores do sistema são o passageiro, que acessa livremente as funcionalidades de consulta e configura seus alertas, e o monitor, ator automatizado responsável por vigiar os pontos e disparar os avisos, conforme a Figura 10.

Figura 10 – Diagrama de casos de uso



Fonte: Autor (2026).

6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

O código-fonte completo encontra-se em repositório público, que reúne o servidor, a interface, os testes, os arquivos de implantação e a documentação técnica:

https://github.com/enzopita/vem-ai

A versão em produção pode ser acessada em https://vemai.enzopita.com. O Quadro 14 relaciona os módulos do servidor e sua extensão. A interface reúne os demais arquivos, em Svelte e TypeScript, organizados em telas, componentes e funções auxiliares.

Quadro 14 – Módulos do servidor

Arquivo	Responsabilidade	Linhas
vemai/central.py	Interface da central, agregação regional e monitor	292
vemai/scraper.py	Requisições aos portais e análise do HTML	320
vemai/catalog.py	Montagem e persistência do catálogo de pontos	182
vemai/models.py	Modelos Pydantic das mensagens	126
vemai/db.py	Persistência SQLite dos alertas e notificações	102
vemai/collector.py	Interface do coletor	90
vemai/region.py	Regras de combinação das respostas	80
vemai/push.py	Envio de Web Push com VAPID	52
vemai/alerts.py	Janela de atividade e regras de notificação	43
tests/	79 testes automatizados (pytest)	1005

Fonte: Autor (2026).

Os trechos a seguir foram reproduzidos literalmente do repositório e correspondem às partes centrais do comportamento distribuído descrito no capítulo 5. O Código 1 implementa as regras de notificação.

Código 1 – Regras de aproximação e de desaparecimento

```
def is_active(sub_time: str, now: datetime, window_min: int = 30) -> bool:
    hour, minute = map(int, sub_time.split(":"))
    day_min = 24 * 60
    diff = (now.hour * 60 + now.minute - (hour * 60 + minute)) % day_min
    if diff > day_min // 2:
        diff -= day_min
    return abs(diff) <= window_min


def detect(prev: StopBoard | None, curr: StopBoard, sub: Subscription) -> list[Alert]:
    alerts = [
        Alert("arriving", p.vehicle, p.line.label, p.eta_min)
        for p in curr.predictions
        if p.line.id == sub.line_id and p.eta_min <= sub.lead_min
    ]
    if prev is not None:
        present = {p.vehicle for p in curr.predictions}
        alerts += [
            Alert("vanished", p.vehicle, p.line.label, p.eta_min)
            for p in prev.predictions
            if p.line.id == sub.line_id
```

```
            and p.eta_min > PASSING_THRESHOLD_MIN
            and p.vehicle not in present
        ]
    return alerts
```

Fonte: Autor (2026), arquivo backend/vemai/alerts.py.

O Código 2 apresenta o espalhamento e coleta: as requisições aos coletores são disparadas simultaneamente, e cada falha é registrada na lista de operadoras indisponíveis em vez de interromper a resposta.

Código 2 – Espalhamento e coleta na central

```
async def collector_get(
    op: str, path: str, params: dict | None = None, timeout: float = COLLECTOR_TIMEOUT_S,
missing_ok: bool = False
):
    try:
        resp = await http.get(f"{collectors[op]}{path}", params=params, timeout=timeout)
        if missing_ok and resp.status_code == 404:
            return None
        resp.raise_for_status()
        return resp.json()
    except (httpx.HTTPError, ValueError) as exc:
        raise CollectorError(op) from exc

async def scatter(path: str, params: dict | None = None) -> tuple[dict, list[str]]:
    """Consulta todos os coletores da região; devolve as respostas por operadora e as que
falharam."""
    outcomes = await asyncio.gather(*(collector_get(op, path, params) for op in collectors),
return_exceptions=True)
    results, unavailable = {}, []
    for op, outcome in zip(collectors, outcomes):
        if isinstance(outcome, CollectorError):
            log.warning("coletor %s indisponível em %s", op, path)
            unavailable.append(op)
        elif isinstance(outcome, BaseException):
            raise outcome
        else:
            results[op] = outcome
    if not results:
        raise HTTPException(503, "nenhum coletor da região está disponível")
    return results, unavailable
```

Fonte: Autor (2026), arquivo backend/vemai/central.py.

O Código 3 apresenta o ciclo do monitor, com o agrupamento por ponto, o descarte dos quadros parciais como referência e o registro da notificação antes do envio.

Código 3 – Ciclo do monitor

```
async def cycle(self) -> None:
    now = self.clock()
    by_stop: dict[str, list[Subscription]] = {}
    for sub in self.db.list_subscriptions():
        if is_active(sub.time, now):
            by_stop.setdefault(sub.stop_id, []).append(sub)

    # Só guardamos o que foi lido por inteiro neste ciclo: um snapshot velho ou parcial
geraria falsos "sumiu".
    current: dict[str, RegionBoard] = {}
    for stop_id, subs in by_stop.items():
        try:
            board = await self.fetch_board(stop_id)
        except Exception:
```

```
            log.exception("falha ao ler o ponto %s", stop_id)
            continue
        # Sem um coletor, os veículos dele somem do quadro; comparar com o anterior os daria
por perdidos.
        complete = not board.unavailable
        if complete:
            current[stop_id] = board
        prev = self.previous.get(stop_id) if complete else None
        for sub in subs:
            for alert in detect(prev, board, sub):
                if self.db.record_notification(sub.id, alert.vehicle, alert.kind,
now.date().isoformat()):
                    await self._push(sub, *describe(alert, sub.stop_name))
    self.previous = current

async def _push(self, sub: Subscription, title: str, body: str) -> None:
    try:
        await self.sender.send(sub, title, body)
    except SubscriptionGone:
        self.db.delete_subscription(sub.id)
    except Exception:
        log.exception("falha ao enviar push da inscrição %s", sub.id)
```

Fonte: Autor (2026), arquivo backend/vemai/central.py.

O Código 4 mostra o mecanismo de idempotência: a inserção condicional na tabela com restrição de unicidade informa se a notificação é inédita.

Código 4 – Registro idempotente de notificação

```
CREATE TABLE IF NOT EXISTS notifications (
    subscription_id INTEGER NOT NULL REFERENCES subscriptions(id) ON DELETE CASCADE,
    vehicle TEXT NOT NULL,
    kind TEXT NOT NULL,
    day TEXT NOT NULL,
    UNIQUE (subscription_id, vehicle, kind, day)
);


def record_notification(self, sub_id: int, vehicle: str, kind: str, day: str) -> bool:
    with closing(self._connect()) as conn, conn:
        cur = conn.execute(
            "INSERT OR IGNORE INTO notifications (subscription_id, vehicle, kind, day)
VALUES (?, ?, ?, ?)",
            (sub_id, vehicle, kind, day),
        )
    return cur.rowcount == 1
```

Fonte: Autor (2026), arquivo backend/vemai/db.py.

O Código 5 apresenta a análise do quadro de previsões retornado pelo portal.

Código 5 – Análise do quadro de previsões

```
def _eta_minutes(cell_text: str) -> int:
    if "CHEGANDO" in cell_text:
        return 0
    m = _MINUTES.search(cell_text)
    if not m:
        raise ValueError(f"previsão desconhecida: {cell_text!r}")
    return int(m.group(1))


def parse_predictions(html: str, stop_id: str) -> StopBoard:
    soup = BeautifulSoup(html, "html.parser")
    predictions = []
    for row in soup.select("table.table_results tr[onclick]"):
        line_id = _LINE_ID.search(row["onclick"])
        label_span = row.select_one("span.link_linha_vlt")
        cells = row.find_all("td")
```

```
        if not (line_id and label_span and len(cells) >= 3):
            continue
        label = next(label_span.stripped_strings)
        vehicle = cells[1].find("strong").get_text(strip=True)
        driver_icon = cells[1].select_one("i.fa-user-alt")
        driver = driver_icon.parent.get_text(strip=True) if driver_icon else ""
        eta = _eta_minutes(cells[2].get_text(" ", strip=True))
        predictions.append(
            Prediction(line=Line(id=line_id.group(1), label=label), vehicle=vehicle,
driver=driver, eta_min=eta)
        )
    return StopBoard(stop_id=stop_id, predictions=predictions,
other_lines=_select_lines(soup.select("#other_linhas a.select_lines")))
```

Fonte: Autor (2026), arquivo backend/vemai/scraper.py.

Por fim, o Código 6 mostra o tratamento da notificação no service worker da aplicação.

Código 6 – Recebimento e toque da notificação no service worker

```
sw.addEventListener('push', (event) => {
    const { title, body, url } = parsePayload(event);
    event.waitUntil(
        sw.registration.showNotification(title, {
            body,
            icon: '/icon-192.png',
            badge: '/icon-192.png',
            data: { url }
        })
    );
});

sw.addEventListener('notificationclick', (event) => {
    event.notification.close();
    const url: string = event.notification.data?.url ?? '/';
    event.waitUntil(
        sw.clients.matchAll({ type: 'window', includeUncontrolled: true }).then((windows) =>
{
            const open = windows.find((w) => new URL(w.url).origin === sw.location.origin);
            if (!open) return sw.clients.openWindow(url);
            // navigate() falha em janelas que o SW não controla (e devolve null fora da
origem); aí só resta focar a que existe.
            return open.navigate(url).then(
                (client) => (client ?? open).focus(),
                () => open.focus()
            );
        })
    );
});
```

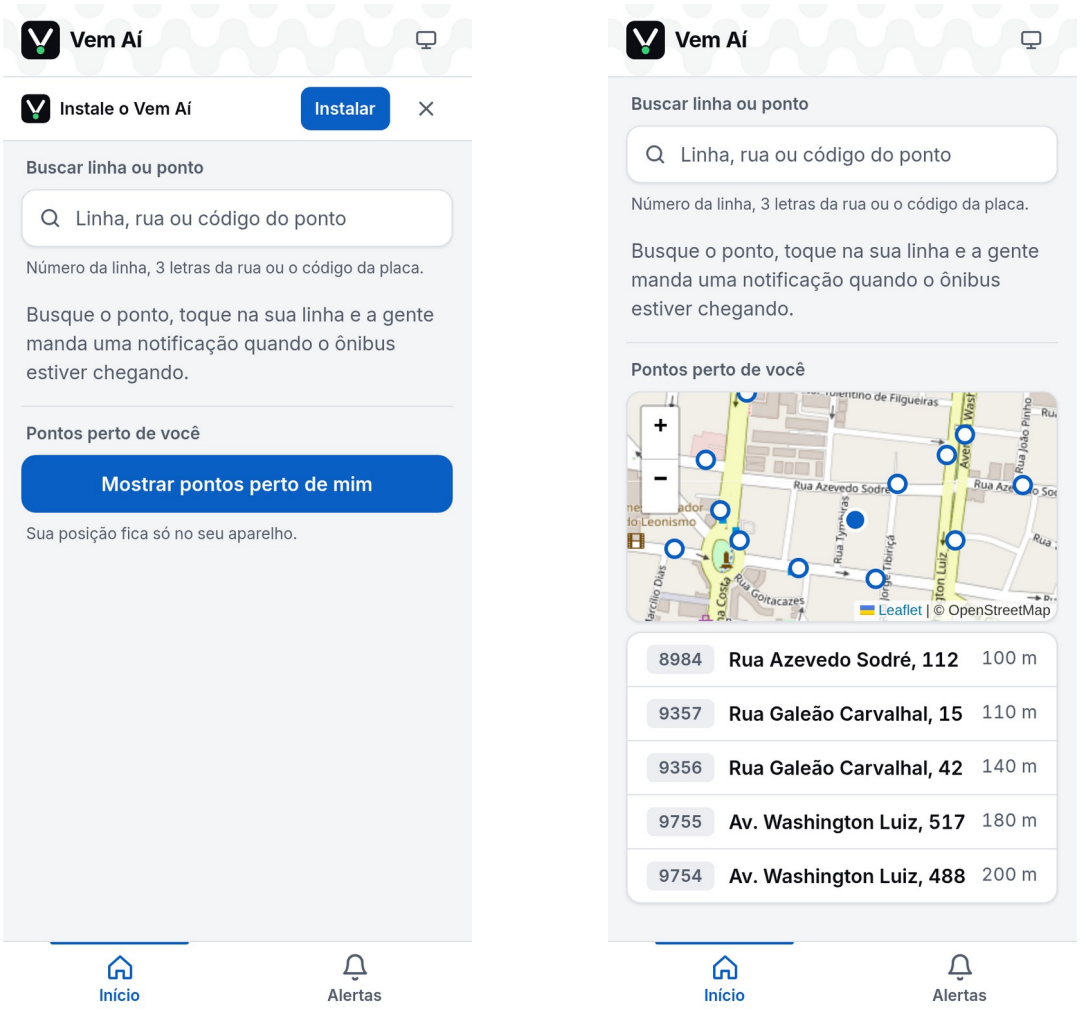
Fonte: Autor (2026), arquivo frontend/src/service-worker/index.ts.

7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO

As capturas a seguir foram obtidas na versão de produção, em https://vemai.enzopita.com, em 26 de outubro de 2026, com o navegador configurado nas dimensões de um telefone celular (390 por 844 pixels). As previsões e as posições exibidas correspondem a dados reais das operadoras no momento da captura. Os primeiros nomes dos condutores, exibidos pela aplicação tal como publicados pelos portais, foram desfocados nas imagens deste documento.

A Figura 11 apresenta a tela inicial. Na primeira abertura, a aplicação oferece a instalação na tela inicial do dispositivo. Ao autorizar a localização, o usuário vê os pontos mais próximos no mapa e em lista, ordenados pela distância calculada no próprio dispositivo.

Figura 11 – Tela inicial (a) e pontos próximos ao usuário (b)

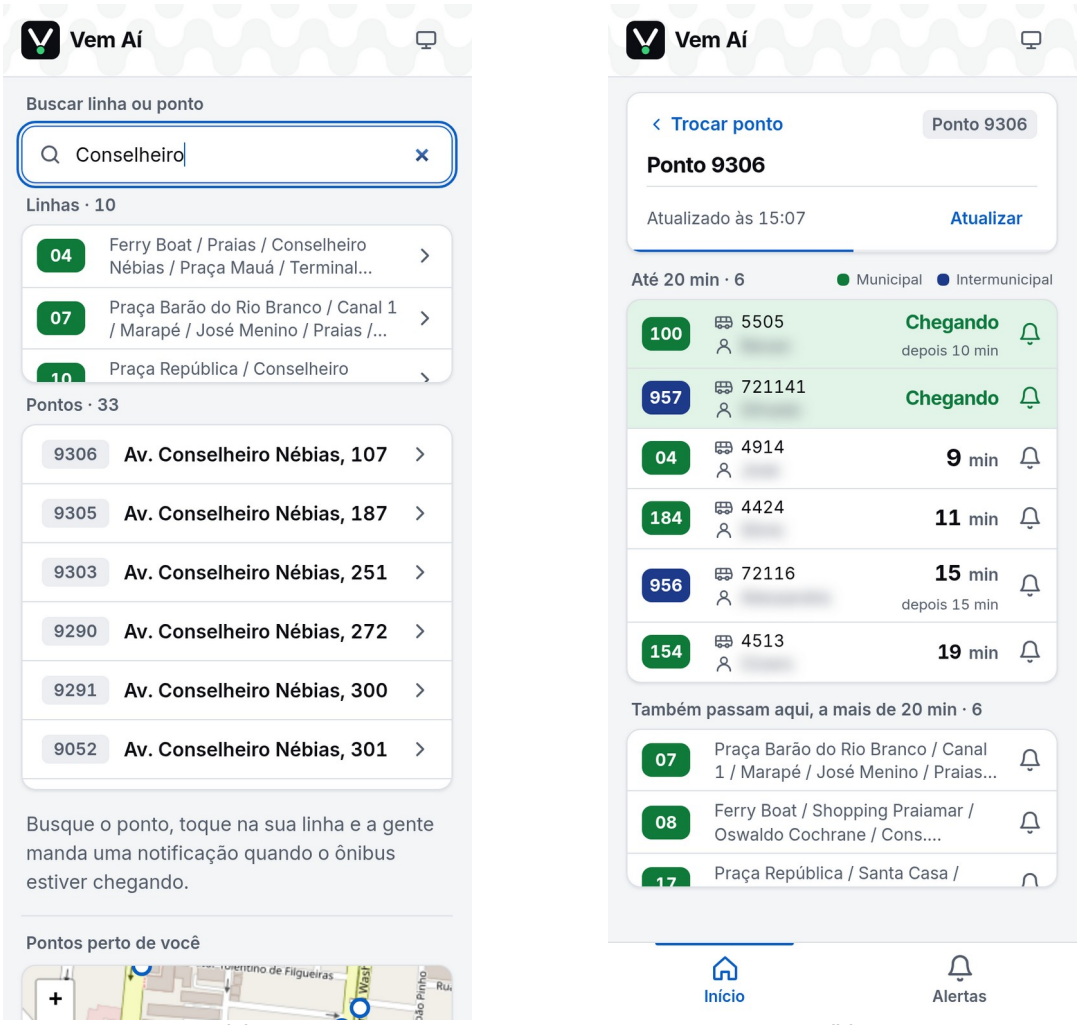


(a) (b)

Fonte: Autor (2026), captura da versão de produção.

A busca aceita o número da linha, parte do nome da rua ou o código impresso na placa do ponto, e retorna linhas e pontos das duas operadoras em conjunto. A tela do ponto, na Figura 12(b), mostra as previsões reunidas das duas operadoras, ordenadas pela chegada, com a linha municipal em verde e a intermunicipal em azul, além das linhas previstas para mais de vinte minutos.

Figura 12 – Resultado da busca (a) e previsões do ponto 9306 (b)

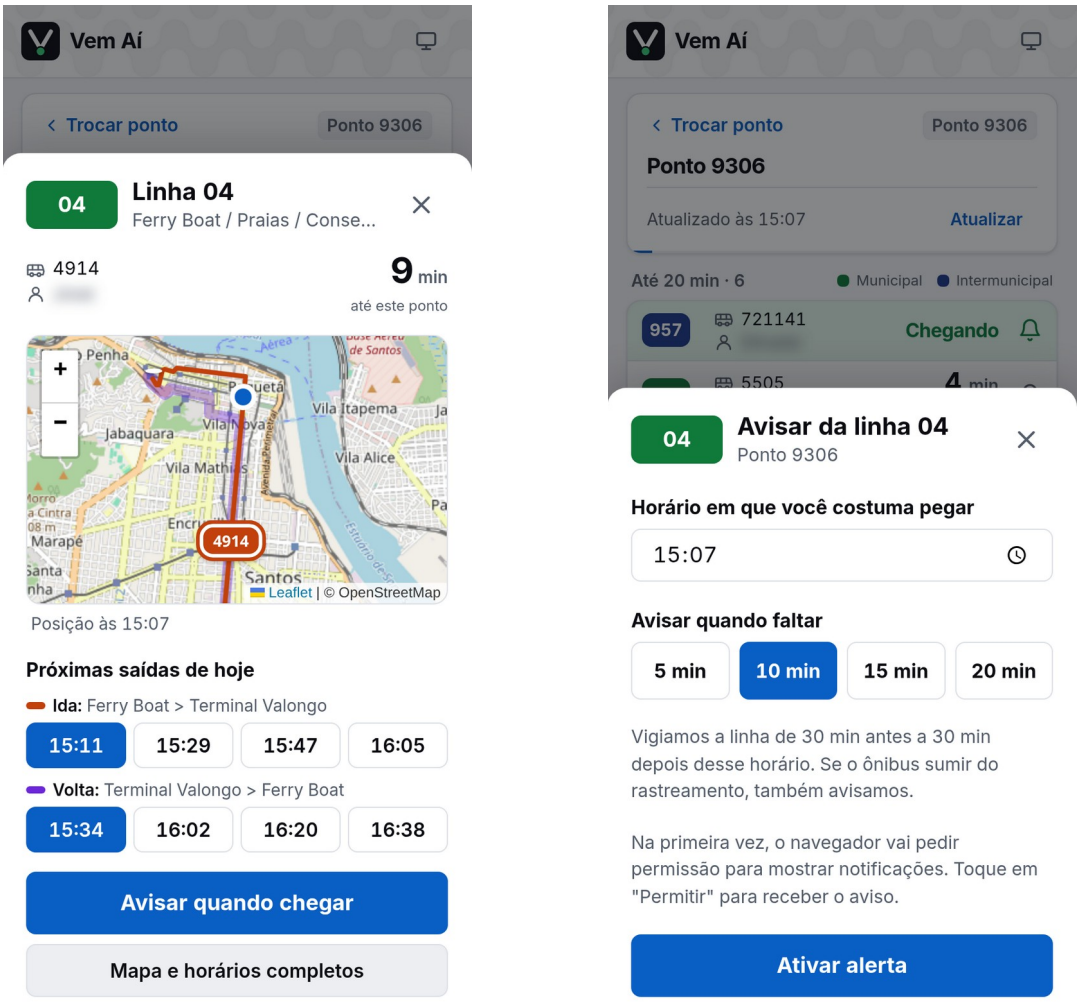


(a) (b)

Fonte: Autor (2026), captura da versão de produção.

Ao tocar em uma previsão, o usuário vê a posição ao vivo daquele veículo sobre o traçado da linha e as próximas saídas programadas. O toque no sino abre a criação do alerta, em que se informam o horário habitual e a antecedência desejada, como mostra a Figura 13.

Figura 13 – Veículo no mapa da linha (a) e criação de alerta (b)

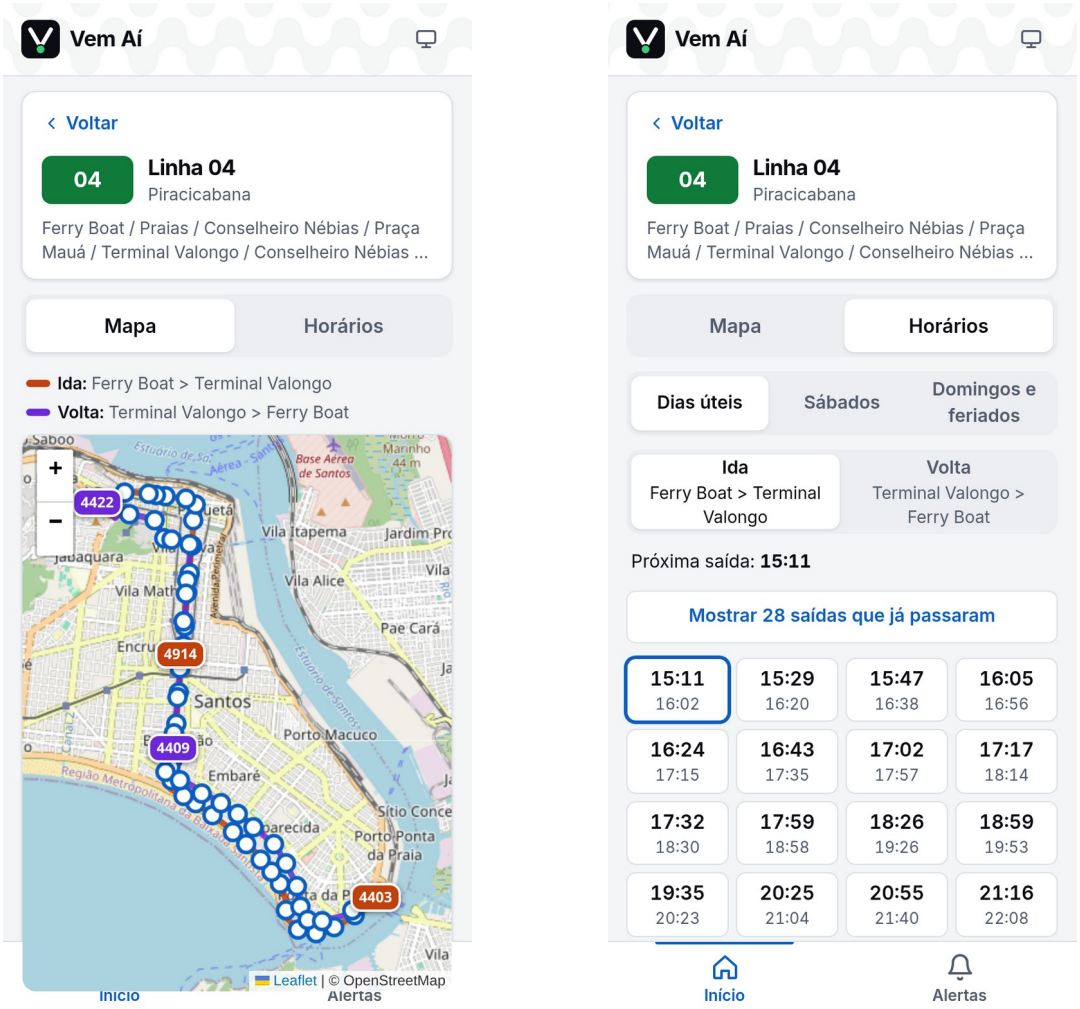


(a) (b)

Fonte: Autor (2026), captura da versão de produção.

A tela da linha, na Figura 14, reúne o traçado de ida e volta, os pontos e todos os veículos em circulação, atualizados a cada quinze segundos, além da tabela horária por tipo de dia e sentido, com destaque para a próxima saída.

Figura 14 – Mapa da linha com veículos ao vivo (a) e tabela horária (b)

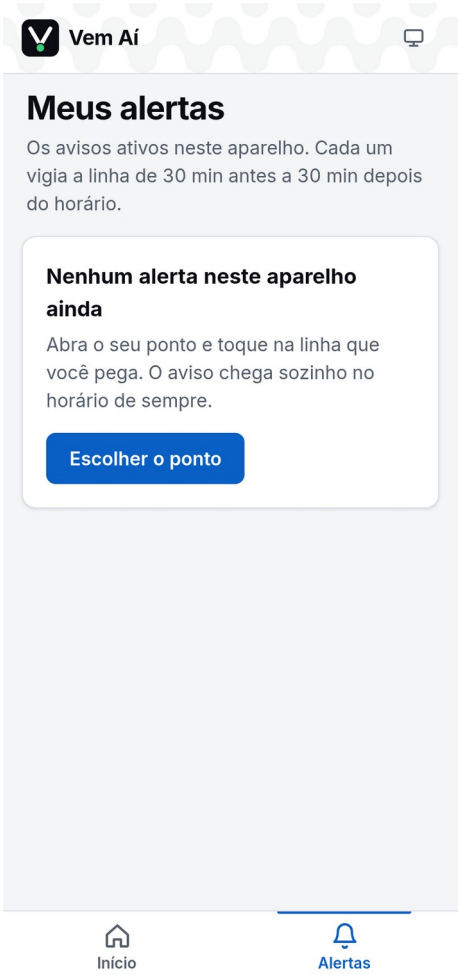


(a) (b)

Fonte: Autor (2026), captura da versão de produção.

A tela de alertas, na Figura 15, lista os avisos registrados no dispositivo, com as opções de enviar uma notificação de teste e de excluir o alerta. A notificação de teste percorre o mesmo caminho de um aviso real, passando pela central e pelo serviço de notificação do navegador.

Figura 15 – Tela de alertas do dispositivo



Fonte: Autor (2026), captura da versão de produção.

A disponibilidade dos nós pode ser verificada pela rota /api/status, que no momento da captura retornou os dois coletores em operação:

```
[{"name":"piracicabana","online":true},
 {"name":"brmobilidadebs","online":true}]
```

A tolerância a falhas é demonstrada interrompendo um dos coletores com o comando “docker compose stop coletor-brmobilidadebs”. A aplicação continua exibindo os pontos e as previsões da outra operadora, acompanhados do aviso de que a BR Mobilidade está fora do ar; o comando “docker compose start” restabelece o serviço sem qualquer intervenção na central ou na aplicação.

8 INTERDISCIPLINARIDADE E CONTRIBUIÇÃO PARA A FORMAÇÃO

O desenvolvimento deste trabalho exigiu a mobilização conjunta de conhecimentos de diferentes disciplinas do curso, o que evidencia o caráter interdisciplinar inerente à construção de sistemas computacionais reais.

8.1 Áreas do conhecimento mobilizadas

A disciplina de Desenvolvimento de Sistemas Distribuídos constituiu o eixo central do projeto. A separação em nós independentes, o espalhamento e coleta, a detecção de falhas por tempo limite, a degradação controlada diante de falhas parciais, a concentração do estado em um único componente e a idempotência do monitor não foram empregados como ilustração teórica, mas como fundamento direto do comportamento do sistema.

Redes de Computadores forneceu os fundamentos dos protocolos empregados, da distinção entre o modelo de requisição e resposta e a comunicação iniciada pelo servidor, da cadeia de certificados TLS, cuja incompletude nos portais precisou ser contornada sem sacrificar a segurança, e do papel do proxy reverso.

Banco de Dados contribuiu para a modelagem das entidades e, sobretudo, para a percepção de que uma restrição de unicidade resolve, de forma atômica e simples, um problema de coordenação que de outro modo exigiria protocolo próprio. Sistemas Operacionais fundamentou o entendimento dos contêineres como processos isolados pelo núcleo, da execução sem privilégios e do modelo de concorrência cooperativa do laço de eventos.

Engenharia de Software orientou o levantamento dos requisitos, a modelagem por diagramas, a definição de interfaces estáveis entre os módulos e a adoção de testes automatizados sobre cópias reais das páginas, condição para alterar o analisador com segurança. A própria revisão do plano, motivada por constatações feitas durante a implementação, constituiu exercício concreto de gestão de mudanças de escopo.

Geoprocessamento foi necessário à montagem do catálogo de pontos com coordenadas, ao cálculo da distância geodésica pela fórmula de haversine e à exibição de traçados e posições sobre mapas. Interação Humano-Computador orientou decisões de interface voltadas ao uso na rua, como contraste elevado,

alvos de toque amplos e, principalmente, a comunicação honesta da incerteza: avisar que um veículo sumiu do rastreamento ou que uma operadora está indisponível, em vez de exibir informação incompleta como se fosse completa.

Por fim, aspectos éticos e jurídicos relacionados à proteção de dados pessoais influenciaram decisões de implementação, entre as quais a dispensa de cadastro, a não persistência da localização do usuário e a não persistência dos nomes dos condutores.

8.2 Contribuição para a formação dos integrantes

A principal contribuição do trabalho consistiu na experiência de projetar um sistema a partir de fontes de dados reais, não controladas pelo grupo e sujeitas a falhas imprevisíveis. Diferentemente de exercícios em que os dados são fornecidos de forma estruturada e estável, o projeto exigiu investigação das fontes, tratamento de formatos irregulares e a formulação de hipóteses sobre o comportamento de sistemas externos cujo funcionamento interno é desconhecido.

O segundo aprendizado refere-se à simplicidade como critério de projeto. A investigação das fontes revelou que parte da complexidade prevista na revisão anterior não era necessária, e a decisão de removê-la, em vez de mantê-la por já ter sido planejada, resultou em um sistema mais fácil de testar, de implantar e de explicar, sem perda dos conceitos de sistemas distribuídos envolvidos.

O terceiro aprendizado refere-se ao tratamento da incerteza. Sistemas distribuídos operam com informação incompleta, e a decisão sobre o que comunicar ao usuário quando parte do sistema não responde revelou-se tão importante quanto o processamento em si. Registra-se, por fim, a experiência de trabalho em equipe com frentes técnicas distintas, que exigiu a definição antecipada de contratos entre os módulos, aproximando a dinâmica do grupo daquela encontrada em ambientes profissionais.

9 CONCLUSÃO

O presente trabalho apresentou o projeto e a implementação do Vem Aí, aplicação distribuída para dispositivos móveis que avisa o passageiro do transporte público da Baixada Santista quando o seu ônibus se aproxima do ponto ou deixa de ser rastreado, desenvolvida no âmbito da disciplina de Desenvolvimento de Sistemas Distribuídos.

Os objetivos estabelecidos foram alcançados. Foi implementada uma arquitetura composta por quatro nós independentes, executados em contêineres e comunicando-se por HTTP com mensagens JSON. Foram desenvolvidos coletores que extraem dos portais das duas operadoras as previsões, os pontos, as linhas, os horários e as posições dos veículos, e uma central que unifica essas respostas em uma visão regional pelo padrão de espalhamento e coleta. O monitor acompanha os pontos com alertas ativos e envia notificações de aproximação e de desaparecimento, sem duplicidade, graças ao registro idempotente apoiado em restrição de unicidade. A falha de um coletor é tratada de forma controlada, tanto nas consultas quanto no monitor. A interface foi construída como aplicação web progressiva, instalável e capaz de receber notificações com a aplicação fechada, e a solução encontra-se em produção, acessível publicamente, com setenta e nove testes automatizados.

A reorientação do projeto em relação à Revisão 02 mostrou-se acertada. A constatação de que os portais já fornecem a previsão de chegada permitiu concentrar o esforço no que os sistemas existentes não oferecem, o aviso no momento certo e a detecção do desaparecimento, e a substituição da plataforma de borda por serviços em contêineres tornou a arquitetura reproduzível em qualquer máquina e mais simples de explicar, sem abandonar os conceitos da disciplina.

Entre as limitações, registra-se a dependência da estrutura dos portais, sujeita a alteração sem aviso, mitigada pelo isolamento e pelos testes do analisador. Registra-se também que a central concentra o estado e constitui ponto único de falha, condição assumida em favor da simplicidade, e que a regra de desaparecimento é heurística: um veículo pode sair da lista por motivos diversos da perda de sinal. Reconhece-se ainda que as previsões dependem da qualidade da estimativa calculada pelas próprias operadoras.

Como trabalhos futuros, identificam-se a produção de indicadores históricos de pontualidade e de disponibilidade a partir das observações do monitor, a replicação da central com banco de dados em servidor, a incorporação de dados em formato padronizado de transporte público, caso venham a ser disponibilizados, e a avaliação sistemática da taxa de falsos avisos de desaparecimento.

Conclui-se que o tema mostrou-se particularmente adequado à disciplina, uma vez que as limitações observadas no serviço existente correspondem a problemas clássicos da área de sistemas distribuídos. A possibilidade de aplicar conceitos teóricos a um problema verificável no cotidiano dos próprios integrantes conferiu ao trabalho caráter formativo que transcende o cumprimento da atividade acadêmica.

GLOSSÁRIO

Quadro 15 – Glossário de termos

Termo	Definição
Contêiner	Processo isolado pelo núcleo do sistema operacional, criado a partir de uma imagem que reúne a aplicação e suas dependências
Coletor	Serviço do sistema responsável pela extração dos dados de uma operadora
Central	Serviço do sistema que agrega as respostas dos coletores, guarda os alertas e envia as notificações
Endpoint	Endereço exclusivo de notificação de um dispositivo, criado pelo serviço do navegador
Espalhamento e coleta	Envio simultâneo de uma requisição a várias fontes, seguido da combinação das respostas
Extração de dados da web	Obtenção automatizada de informações publicadas em páginas destinadas à leitura humana
Idempotência	Propriedade de uma operação cuja repetição produz o mesmo efeito que uma execução única
Laço de eventos	Mecanismo que alterna a execução de tarefas assíncronas enquanto elas aguardam entrada e saída
Proxy reverso	Servidor que recebe requisições externas e as encaminha internamente ao serviço adequado
PWA	Aplicação web progressiva; aplicação web que incorpora recursos de aplicativo nativo
Quadro de previsões	Conjunto das previsões de chegada de um ponto em determinado instante
Service worker	Script executado em segundo plano pelo navegador, capaz de interceptar requisições e receber notificações
Sondagem periódica	Consulta repetida ao servidor em intervalos regulares para verificar alterações
Tempo limite	Intervalo máximo de espera por uma resposta, após o qual o componente é considerado indisponível
Transparência	Ocultação, ao usuário, da natureza distribuída do sistema
VAPID	Esquema de identificação do servidor de aplicação junto aos serviços de notificação
Web Push	Padrão de envio de notificações do servidor ao navegador, mesmo com a página fechada

Fonte: Autor (2026).

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6024**: informação e documentação: numeração progressiva das seções de um documento: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6027**: informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014 (Marco Civil da Internet). **Diário Oficial da União**: seção 1, Brasília, DF, 15 ago. 2018.

CHANDRA, Tushar Deepak; TOUEG, Sam. Unreliable failure detectors for reliable distributed systems. **Journal of the ACM**, New York, v. 43, n. 2, p. 225-267, mar. 1996.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

DOCKER. **Docker overview**. Disponível em: https://docs.docker.com/get-started/docker-overview/. Acesso em: 26 out. 2026.

EMTU – EMPRESA METROPOLITANA DE TRANSPORTES URBANOS DE SÃO PAULO. **Quem somos**: Baixada Santista. Disponível em: https://www.emtu.sp.gov.br/emtu/institucional/quem-somos/baixada-santista.fss. Acesso em: 14 set. 2026.

FISCHER, Michael J.; LYNCH, Nancy A.; PATERSON, Michael S. Impossibility of distributed consensus with one faulty process. **Journal of the ACM**, New York, v. 32, n. 2, p. 374-382, abr. 1985.

HOHPE, Gregor; WOOLF, Bobby. **Enterprise integration patterns**: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

IBGE – INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Estimativas da população residente nos municípios brasileiros**. Rio de Janeiro: IBGE, 2026. Disponível em: https://www.ibge.gov.br/estatisticas/sociais/populacao.html. Acesso em: 14 set. 2026.

KUROSE, James F.; ROSS, Keith W. **Redes de computadores e a Internet**: uma abordagem top-down. 6. ed. São Paulo: Pearson, 2013.

PYTHON SOFTWARE FOUNDATION. **asyncio**: asynchronous I/O. Disponível em: https://docs.python.org/3/library/asyncio.html. Acesso em: 26 out. 2026.

RICHARDSON, Leonard. **Beautiful Soup documentation**. Disponível em: https://www.crummy.com/software/BeautifulSoup/bs4/doc/. Acesso em: 26 out. 2026.

SÃO PAULO (Estado). Lei Complementar nº 815, de 30 de julho de 1996. Cria a Região Metropolitana da Baixada Santista e dá providências correlatas. **Diário Oficial do Estado de São Paulo**, São Paulo, 31 jul. 1996.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.

THOMSON, Martin. **Message encryption for Web Push**. RFC 8291. Internet Engineering Task Force, nov. 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8291. Acesso em: 26 out. 2026.

THOMSON, Martin; BECK, Peter. **Voluntary Application Server Identification (VAPID) for Web Push**. RFC 8292. Internet Engineering Task Force, nov. 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8292. Acesso em: 14 set. 2026.

THOMSON, Martin; DAMAGGIO, Elio; RAYMOR, Brian. **Generic event delivery using HTTP push**. RFC 8030. Internet Engineering Task Force, dez. 2016. Disponível em: https://www.rfc-editor.org/rfc/rfc8030. Acesso em: 14 set. 2026.

W3C – WORLD WIDE WEB CONSORTIUM. **Service Workers**. 2026a. Disponível em: https://www.w3.org/TR/service-workers/. Acesso em: 14 set. 2026.

W3C – WORLD WIDE WEB CONSORTIUM. **Web Application Manifest**. 2026b. Disponível em: https://www.w3.org/TR/appmanifest/. Acesso em: 14 set. 2026.